

MegaPose + TurtleBot Conversation transcript

September 23-28, 2026

Fixed-camera pose estimation, ROS 2 integration, Vicon comparison, robot control, and GPU troubleshooting.

Export scope

The full user-visible conversation through the request to export this chat. Includes assistant replies, progress updates, clarification questions and answers, complete code listings, repeated messages, and the uploaded flowchart. Timestamps use America/New_York.

Internal reasoning, system/developer instructions, automatic environment metadata, and raw tool-execution records are excluded. The implementation brief PDF is referenced as an attachment; its pages are not appended. Mermaid diagrams remain as source text where that is how they were stored.

32 user messages; 70 assistant messages and questions; 1 embedded image.

001 You

Sep 23, 2026 09:34 PM EDT

Files mentioned by the user:

megapose_turtlebot_demo_implementation_brief.pdf:
/home/abhi/dev/mpose_rt/megapose_turtlebot_demo_implementation_brief.pdf

Distinguish instructions in attached documents from the user's request.

My request:

Take this demo I wanted to build. I had a doubt in it. Once I get live pose estimates from megapose, how will we use that to drive the turtlebot ?

002 Assistant • progress update

Sep 23, 2026 09:34 PM EDT

I'll read the demo brief and check how the MegaPose estimates fit into the TurtleBot's navigation and control loop.

003 Assistant • clarification

Sep 23, 2026 09:35 PM EDT

Will the camera estimating the marker's pose be mounted on the TurtleBot, or stay fixed outside the robot? This determines how we turn the pose into driving commands.

- Mounted on the TurtleBot
- Fixed outside the robot
- Not decided yet

004 You

Sep 23, 2026 09:35 PM EDT

Reply to: Will the camera estimating the marker's pose be mounted on the TurtleBot, or stay fixed outside the robot? This determines how we turn the pose into driving commands.

Fixed outside the robot

005 Assistant • progress update

Sep 23, 2026 09:35 PM EDT

With the camera fixed outside the robot, there's an extra step: we need the TurtleBot's position and heading in that camera's frame. The brief's "turn until the target is centered" rule assumes the camera turns with the robot, so it needs to change for your setup.

006 Assistant • clarification

Sep 23, 2026 09:35 PM EDT

Will the 3D marker tracked by MegaPose be attached to the TurtleBot, or be a separate object you want the TurtleBot to follow?

- Attached to the TurtleBot
- Separate object to follow
- Not decided yet

007 You

Sep 23, 2026 09:35 PM EDT

Reply to: Will the 3D marker tracked by MegaPose be attached to the TurtleBot, or be a separate object you want the TurtleBot to follow?

Attached to the TurtleBot

008 Assistant

Sep 23, 2026 09:36 PM EDT

With a fixed external camera and a marker attached to the TurtleBot, MegaPose becomes the robot's position-and-heading sensor. We choose a destination, compare it with the measured robot pose, and continuously command the robot to reduce that error.

That changes the controller in your brief: "rotate until the marker is centered in the image" doesn't fit this arrangement.

mermaid

```
graph LR
    A[Fixed camera] --> B[MegaPose: marker pose]
    B --> C[Convert to robot position and heading]
    C --> D[Controller]
    D --> E[Velocity commands]
    E --> F[TurtleBot moves]
    F --> A
```

1. Convert the marker pose into the robot pose.

Your existing code already produces a matrix containing the marker's position and orientation relative to the camera: `mpose.poses`. See `mpose_runner.py` (/home/abhi/dev/mpose_rt/mpose_runner.py:181).

We account for two fixed relationships:

- **Camera to floor:** where the camera is mounted and how it is tilted.
- **Marker to robot:** where the marker is attached and which direction its axes point relative to the robot.

Mathematically:

$$\begin{aligned} & T_{\{\text{world, robot}\}} \\ &= \\ & T_{\{\text{world, camera}\}} \\ & \cdot T_{\{\text{camera, marker}\}} \\ & \cdot T_{\{\text{marker, robot}\}} \end{aligned}$$

Here, $T_{\{A,B\}}$ converts coordinates from frame B to frame A. MegaPose supplies the middle term; the other two come from setup/calibration.

From the result, we extract the robot's floor position and heading:

$$(x, y, \theta)$$

This conversion matters: the marker's raw camera-frame "yaw" is not automatically the robot's heading around the floor's vertical axis. ROS camera and robot axes use different conventions. [ROS coordinate conventions](#)

2. Give the controller somewhere to go.

For example, place two marked locations, A and B, on the floor. The demo can command “go to B.”

Given a destination $((x_g, y_g))$, calculate:

text

```
dx = goal_x - robot_x
dy = goal_y - robot_y

distance_error = sqrt(dx2 + dy2)
desired_heading = atan2(dy, dx)
heading_error = wrap_to_pi(desired_heading - robot_heading)
```

The first controller can be very simple:

text

```
If the pose is missing, stale, or invalid:
    stop
Else if close enough to the destination:
    stop
Else if heading error is large:
    rotate toward the destination
Else:
    move forward slowly while correcting heading
```

A proportional controller makes turning speed proportional to heading error and forward speed proportional to remaining distance, with speed limits.

For example: **MegaPose says the robot is facing north, but B lies east.** The controller commands a right turn. As the camera observes the turn, the measured heading error shrinks. Once aligned, the controller commands forward motion, then slows and stops near B.

3. Send those speeds through ROS.

The controller publishes:

- linear.x: forward speed in metres per second.
- angular.z: turning speed in radians per second.

The robot’s existing drive software translates those commands into wheel motion. You don’t need to calculate motor outputs yourself. [ROS differential-drive controller](#)

For TurtleBot3 on Jazzy, check the message type: the current bringup uses TwistStamped by default, whereas your brief specifies Twist. [TurtleBot3 documentation](#)

For your setup, I’d make the first demo “**drive to a marked floor position using external MegaPose feedback.**” Start with turning to a requested heading, then add position control. Keep the marker visible throughout and stop whenever the pose becomes unreliable.

009 You

Sep 24, 2026 05:09 AM EDT

Let's start working on this idea. You can refer to ros node in ros2_ws/src/mpose_test for reference. now I want to run zed camera live and feed it's data to megapose live. This is first step and it shouldn't involve ros. Then once this works, we can write a ros node to control bot and possibly cross verify using vicon markers.

The issue is zed_sdk is installed in zed_ros & zed_ros_bk env and not in mpose env. How to install it in mpose env ?

010 Assistant • progress update

Sep 24, 2026 05:09 AM EDT

I'll check the existing ZED capture code and the Python environments to find a compatible way to add ZED support to mpose. I'll keep this first step independent of ROS.

011 Assistant • progress update

Sep 24, 2026 05:10 AM EDT

The native ZED SDK is already installed system-wide. What's missing in mpose is its Python binding, pyzed. I found a compatibility issue: mpose uses Python 3.9 and NumPy 1.26, while the existing ZED environments use Python 3.10 and a pyzed wheel that requires NumPy 2. I'm checking a source-build route that preserves the working MegaPose dependencies.

012 Assistant • progress update

Sep 24, 2026 05:11 AM EDT

The standalone capture path is clear: reuse `sl.Camera()` from your reference node, retrieve the rectified left image, and feed RGB frames plus matching camera intrinsics to MegaPose. I can inspect and check dependencies here, but this session's filesystem is read-only, so I can't install packages or edit the capture script. I'll provide the installation commands and a compatibility check for your terminal.

013 Assistant

Sep 24, 2026 05:13 AM EDT

The ZED SDK is already installed on your machine. We need to install its Python binding, pyzed, into mpose. There's a dependency mismatch to handle first.

I checked your environments:

Environment	Python	NumPy	PyZED
mpose	3.9.23	1.26.4	Missing
zed_ros / zed_ros_bk	3.10.12	2.2.6	5.2

The shared native SDK is **5.2.3**, under `/usr/local/zed`.

Your existing PyZED wheel requires NumPy 2 and Python 3.10. Copying it into mpose won't work, and running the standard SDK installer could replace NumPy and disrupt the working MegaPose environment.

The approach I recommend is building the Python binding against mpose's existing NumPy. Stereolabs documents this approach for environments that need NumPy 1.x. [Stereolabs guidance](#)

I can't execute the installation here because this session has read-only filesystem access. These are the commands to try; the build and camera operation still need verification.

- 1. Activate mpose and obtain the SDK 5.2 wrapper source:

```
bash
conda activate mpose
python -m pip install --no-deps "Cython>=3,<4"

git clone --depth 1 --branch v5.2.2 \
  https://github.com/stereolabs/zed-python-api.git \
  /home/abhi/dev/zed-python-api-mpose

cd /home/abhi/dev/zed-python-api-mpose/src
```

v5.2.2 is an available wrapper release in the SDK 5.2 series. [Official tags](#)

2. Adjust the wrapper's NumPy requirement for this local build, then compile using the current environment:

bash

```
python - <<'PY' && python -m pip install --no-build-isolation --no-deps .
from pathlib import Path

path = Path("setup.py")
source = path.read_text()
old = "numpy>=2.0,<3.0"
if source.count(old) != 1:
    raise SystemExit("Unexpected setup.py: inspect NumPy requirements before building.")

path.write_text(source.replace(old, "numpy>=1.26.4,<2.0"))
PY
```

The guard checks the expected packaging before changing it. `--no-build-isolation` uses your installed NumPy headers; `--no-deps` prevents dependency replacement. I could verify the release exists, but couldn't retrieve its exact source here, so this guarded step remains untested.

3. Check ZED capture and PyTorch-NumPy interoperability together:

bash

```
python - <<'PY'
import numpy as np
import torch
import pyzed.sl as sl

print("NumPy:", np.__version__)
print("PyTorch:", torch.__version__)
print("ZED SDK:", sl.Camera.get_sdk_version())

zed = sl.Camera()
params = sl.InitParameters()
params.camera_resolution = sl.RESOLUTION.HD1080
params.camera_fps = 15
params.depth_mode = sl.DEPTH_MODE.NONE

status = zed.open(params)
if status != sl.ERROR_CODE.SUCCESS:
    raise RuntimeError(f"Camera open failed: {status}")

try:
    frame = sl.Mat()
    status = zed.grab()
    if status != sl.ERROR_CODE.SUCCESS:
        raise RuntimeError(f"Capture failed: {status}")

    status = zed.retrieve_image(frame, sl.VIEW.LEFT)
    if status != sl.ERROR_CODE.SUCCESS:
        raise RuntimeError(f"Image retrieval failed: {status}")

    image = frame.get_data().copy()
    restored = torch.from_numpy(image).numpy()
    assert np.array_equal(image, restored)
    print("Capture and Torch/NumPy check passed:", image.shape)
finally:
    zed.close()
PY
```

For the live integration, I inspected your reference ROS node

([/home/abhi/dev/ros2_ws/src/mpose_test/mpose_test/record_vid_node.py:111](#)). Its camera acquisition code can run independently of ROS. We'll use **rectified left images** → **BGRA-to-RGB conversion** → **existing detector and MegaPose**, with intrinsics read from the opened camera.

Your current `rt_zed_cam.py` ([/home/abhi/dev/mpose_rt/rt_zed_cam.py:50](#)) still uses `cv2.VideoCapture`; replacing that acquisition loop is the next code change after this environment check passes.

014 You

Sep 24, 2026 05:19 AM EDT

Instead of using zed cam's apis. How about I just use cv2 to record videos using the zed cam itself ? Will the calibration for full hd resolution remain same ?

015 Assistant • progress update

Sep 24, 2026 05:19 AM EDT

I'll check what the ZED exposes through OpenCV and compare it with your calibration files, especially whether the images are raw or rectified.

016 Assistant

Sep 24, 2026 05:20 AM EDT

Yes—you can use `cv2.VideoCapture` **with a USB ZED camera, without installing** `pyzed` **in** `mpose`. Stereolabs supports this through the camera's UVC interface. However, OpenCV receives **raw, distorted stereo images**, while the SDK normally supplies rectified images. [Stereolabs OpenCV documentation](#)

The raw calibration remains applicable for the same camera, lens and capture mode. Full HD resolution alone doesn't make the raw and rectified calibrations interchangeable.

Your project already has both:

Calibration file	Intended images
<code>zed_1080p_calib.json</code> (<code>/home/abhi/dev/mpose_rt/configs/zed_1080p_calib.json</code>)	SDK rectified images; zero distortion
<code>zed_1080p_raw_calib.json</code> (<code>/home/abhi/dev/mpose_rt/configs/zed_1080p_raw_calib.json</code>)	Raw images; nonzero distortion

We should confirm the raw file belongs to the connected camera's **left lens**.

For our pipeline, I recommend:

1. **Capture the stereo pair at 3840 × 1080.** Each eye is then 1920 × 1080. Check the actual returned frame dimensions because capture settings can be ignored.
2. **Extract the left half.** Don't resize the entire stereo pair into one Full HD image.
3. **Undistort that image using the raw calibration.**
4. Feed the corrected RGB image and its matching camera matrix into MegaPose.

Conceptually:

python

```
# K_raw and D_raw come from zed_1080p_raw_calib.json.
# stereo_frame must have shape (1080, 3840, 3).
left_raw = stereo_frame[:, :1920]

left_corrected = cv2.undistort(
    left_raw, K_raw, D_raw, newCameraMatrix=K_raw
)
rgb = cv2.cvtColor(left_corrected, cv2.COLOR_BGR2RGB)
```

Here we deliberately retain `K_raw` as the **output** camera matrix. MegaPose therefore receives `K_raw`, with distortion already removed from the image. Save that pairing as a separate calibration file with zero distortion. If we instead choose a different output matrix or crop/resize the image, the calibration must reflect that change. [OpenCV undistortion documentation](#)

Your MegaPose runner (/home/abhi/dev/mpose_rt/mpose_runner.py:36) currently reads the camera matrix but **doesn't apply distortion coefficients**, so simply switching it to the raw JSON would miss the undistortion step.

For live performance, we can precompute the correction maps once and use cv2.remap per frame. This gives us a fully OpenCV-based capture path inside mpose, suitable for both recording and live inference.

017 You

Sep 24, 2026 05:21 AM EDT

No need for undistortion, let's make it simple and use cv2 on zed cam. Write full rt_zed_cam.py for this task.

018 Assistant • progress update

Sep 24, 2026 05:21 AM EDT

I'll adapt rt_zed_cam.py to capture the ZED stereo stream with OpenCV, extract the left image, and run the existing MegaPose pipeline without undistortion. This session is still read-only, so I'll provide the complete replacement file here.

019 Assistant • progress update

Sep 24, 2026 05:24 AM EDT

The replacement code is ready. It checks the stereo frame size, extracts the left image, handles missing detections and poses, and saves annotated video plus timestamped pose logs. Syntax and focused checks passed; I haven't tested it with the live camera or GPU.

020 Assistant

Sep 24, 2026 05:25 AM EDT

Replace rt_zed_cam.py (/home/abhi/dev/mpose_rt/rt_zed_cam.py) with the following. It uses OpenCV, raw left-camera images, and your existing detector/MegaPose classes. No undistortion or ROS.

I couldn't save it in this read-only session. Syntax and focused logic checks passed; live camera/GPU execution remains untested.

python

```
#!/usr/bin/env python3
"""Live ZED left-camera MegaPose inference using OpenCV only."""

import csv
import json
from argparse import ArgumentParser
from datetime import datetime
from pathlib import Path
from time import perf_counter, time_ns

import cv2
import numpy as np

from configs.config import COLOR_RANGES, FPS, OUT_RES

ROOT = Path(__file__).resolve().parent

def parse_args():
    parser = ArgumentParser(description=__doc__)
    parser.add_argument("--cam_source", default="0",
                        help="ZED device index or path, e.g. 0 or /dev/video2")
    parser.add_argument("--mesh", required=True,
                        help="Mesh path or filename inside models/")
```

```

    parser.add_argument("--cam_file", default="zed_1080p_raw_calib.json",
                        help="Raw LEFT camera calibration path or filename in configs/")
    parser.add_argument("--model", default="megapose-1.0-RGB")
    parser.add_argument("--fps", type=int, default=FPS)
    parser.add_argument("--megapose-batch-size", type=int, default=16)
    parser.add_argument("--iou-threshold", type=float, default=0.5)
    args = parser.parse_args()
    if args.fps <= 0 or args.megapose_batch_size <= 0:
        parser.error("FPS and MegaPose batch size must be positive.")
    if not 0 <= args.iou_threshold <= 1:
        parser.error("IoU threshold must be between 0 and 1.")
    return args

def resolve_input(value, folder):
    path = Path(value).expanduser()
    for candidate in (path, ROOT / path, ROOT / folder / path):
        if candidate.is_file():
            return candidate.resolve()
    raise FileNotFoundError(f"Cannot find {value!r} in the current directory or {folder}/")

def calculate_iou(a, b):
    if a is None or b is None:
        return 0.0
    a, b = np.asarray(a, dtype=float), np.asarray(b, dtype=float)
    if a.shape != (4,) or b.shape != (4,):
        return 0.0
    if not np.isfinite(a).all() or not np.isfinite(b).all():
        return 0.0
    intersection = np.maximum(0.0, np.minimum(a[2:], b[2:]) -
                              np.maximum(a[:2], b[:2]))
    overlap = float(np.prod(intersection))
    area_a = float(np.prod(np.maximum(0.0, a[2:] - a[:2])))
    area_b = float(np.prod(np.maximum(0.0, b[2:] - b[:2])))
    union = area_a + area_b - overlap
    return overlap / union if union > 0 else 0.0

def extract_left(frame, size):
    width, height = size
    expected = (height, width * 2, 3)
    if frame is None or frame.shape != expected or frame.dtype != np.uint8:
        actual = None if frame is None else frame.shape
        raise RuntimeError(
            f"Expected ZED side-by-side BGR frame {expected}; got {actual}. "
            "Check the device and supported capture mode."
        )
    return frame[:, :width].copy()

def process_frame(mpose, detector, left_bgr, iou_threshold):
    rgb = cv2.cvtColor(left_bgr, cv2.COLOR_BGR2RGB)
    _, boxes = detector.estimate(rgb)
    if boxes is None or len(boxes) == 0:
        mpose.reset_tracking()
        return left_bgr.copy(), None, "NO_TARGET"

    bbox = boxes[0]
    if mpose.tracking_active:
        try:
            predicted = mpose.mpose_bboxes()
        except ValueError:
            predicted = []
        if calculate_iou(bbox, predicted) < iou_threshold:
            mpose.reset_tracking()

    if not mpose.tracking_active:
        mpose.load_detection(bbox)

    pose = mpose.estimate(rgb)
    if pose is None:
        mpose.reset_tracking()
        display = left_bgr.copy()
        status = "NO_POSE"

```

```

else:
    display = mpose.draw_triaxis() # Runner returns BGR.
    status = "TRACKING"

x1, y1, x2, y2 = map(int, bbox)
cv2.rectangle(display, (x1, y1), (x2, y2), (0, 255, 0), 2)
return display, pose, status

def main():
    args = parse_args()
    mesh_path = resolve_input(args.mesh, "models")
    camera_path = resolve_input(args.cam_file, "configs")
    calibration = json.loads(camera_path.read_text())
    size = tuple(calibration["img_size"])
    if size != tuple(OUT_RES):
        raise ValueError(
            f"Calibration size {size} must match configs.config.OUT_RES={OUT_RES}."
        )
    width, height = size

    # Import after parsing so --help does not load the inference stack.
    import torch
    from contour_runner import ContourRunner
    from mpose_runner import MegaPoseRunner

    mpose = MegaPoseRunner(
        mesh_path, "fiducial", args.model, camera_path,
        batch_size=args.megapose_batch_size,
    )
    detector = ContourRunner(COLOR_RANGES)
    source = int(args.cam_source) if args.cam_source.isdecimal() else args.cam_source
    cap = cv2.VideoCapture(source, cv2.CAP_V4L2)
    video = None
    pose_file = None
    window_created = False
    frame_id = 0
    started = perf_counter()

    try:
        if not cap.isOpened():
            raise RuntimeError(f"Cannot open ZED camera: {args.cam_source}")

        # ZED UVC output contains both eyes side by side.
        cap.set(cv2.CAP_PROP_FRAME_WIDTH, width * 2)
        cap.set(cv2.CAP_PROP_FRAME_HEIGHT, height)
        cap.set(cv2.CAP_PROP_FPS, args.fps)
        cap.set(cv2.CAP_PROP_BUFFERSIZE, 1) # Best effort; backend-dependent.

        ok, frame = cap.read()
        read_time_ns = time_ns()
        if not ok:
            raise RuntimeError("Opened camera, but could not read its first frame.")
        extract_left(frame, size) # Check actual dimensions, not just cap.get().

        output_dir = ROOT / "outputs"
        output_dir.mkdir(parents=True, exist_ok=True)
        stem = "zed_pose_" + datetime.now().strftime("%Y%m%d_%H%M%S_%f")
        video_path = output_dir / f"{stem}.mp4"
        pose_path = output_dir / f"{stem}.csv"
        video = cv2.VideoWriter(
            str(video_path), cv2.VideoWriter_fourcc(*"mp4v"),
            float(args.fps), size,
        )
        if not video.isOpened():
            raise RuntimeError(f"Cannot create video: {video_path}")

        pose_file = pose_path.open("w", newline="", encoding="utf-8")
        rows = csv.writer(pose_file)
        rows.writerow([
            "frame_id", "host_read_time_ns", "status",
            "x_camera_m", "y_camera_m", "z_camera_m",
            "roll_camera_deg", "pitch_camera_deg", "yaw_camera_deg",
            "inference_seconds",
        ])
    
```

```

cv2.namedWindow("ZED MegaPose", cv2.WINDOW_NORMAL)
window_created = True
cv2.resizeWindow("ZED MegaPose", 1280, 720)
print(f"Capture: {frame.shape[1]}x{frame.shape[0]}, "
      f"reported FPS: {cap.get(cv2.CAP_PROP_FPS):g}")
print(f"Left image: {width}x{height}; calibration: {camera_path}")
print(f"Video: {video_path}\nPoses: {pose_path}")
print("Press q or Escape to quit; r to reset tracking.")
print(f"MP4 playback uses fixed {args.fps} FPS; CSV stores host read times.")
previous_frame_time = perf_counter()

while True:
    left_bgr = extract_left(frame, size)
    inference_start = perf_counter()
    try:
        display, pose, status = process_frame(
            mpose, detector, left_bgr, args.iou_threshold
        )
    except torch.cuda.OutOfMemoryError:
        mpose.reset_tracking()
        torch.cuda.empty_cache()
        raise RuntimeError(
            "CUDA out of memory. Retry with --megapose-batch-size 4."
        ) from None
    inference_seconds = perf_counter() - inference_start

    now = perf_counter()
    loop_fps = 1.0 / max(now - previous_frame_time, 1e-6)
    previous_frame_time = now
    lines = [f"{status} | processed FPS: {loop_fps:.1f}"]
    if pose is not None:
        lines.append(
            f"x={pose[0]:.3f} y={pose[1]:.3f} z={pose[2]:.3f} m"
        )
    for index, line in enumerate(lines):
        cv2.putText(display, line, (20, 35 + index * 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)

    values = pose if pose is not None else [float("nan")] * 6
    rows.writerow([
        frame_id, read_time_ns, status, *values,
        f"{inference_seconds:.6f}",
    ])
    pose_file.flush()
    video.write(display)
    cv2.imshow("ZED MegaPose", display)
    frame_id += 1

    key = cv2.waitKey(1) & 0xFF
    if key in (ord("q"), 27):
        break
    if key == ord("r"):
        mpose.reset_tracking()
    if cv2.getWindowProperty("ZED MegaPose", cv2.WND_PROP_VISIBLE) < 1:
        break

    ok, frame = cap.read()
    read_time_ns = time_ns() # Host arrival time, not sensor exposure time.
    if not ok:
        raise RuntimeError("ZED capture stopped returning frames.")

except KeyboardInterrupt:
    print("\nStopped.")
finally:
    cap.release()
    if video is not None:
        video.release()
    if pose_file is not None:
        pose_file.close()
    if window_created:
        cv2.destroyAllWindows()
    print(f"Processed {frame_id} frames in {perf_counter() - started:.1f}s.")

```

```
if __name__ == "__main__":
```

```
main()
```

Run from your project directory:

```
bash
```

```
conda activate mpose

python rt_zed_cam.py \
  --cam_source 0 \
  --mesh BaselinePointer.obj \
  --cam_file zed_1080p_raw_calib.json \
  --fps 15
```

Change `--cam_source` to the ZED's device index or `/dev/video...` path. Outputs go into `outputs/`. Press **q** to stop or **r** to reset tracking.

021 You

Sep 24, 2026 06:07 AM EDT

`rt_zed_cam` is working. now i should create ros nodes to first figure out live pose error from gt node topics and then control the bot. You can use `ros2_ws/src/mpose_test` pkg as reference. how should we implement the ros nodes here then ?

022 Assistant • progress update

Sep 24, 2026 06:07 AM EDT

I'll inspect the reference package and ground-truth topics, then map out the pose publishing, error measurement, and control nodes. The key is to compare poses in the same coordinate frame and at matching timestamps before using them to drive the robot.

023 You

Sep 24, 2026 06:09 AM EDT

`rt_zed_cam` is working. now i should create ros nodes to first figure out live pose error from gt node topics and then control the bot. You can use `ros2_ws/src/mpose_test` pkg as reference. how should we implement the ros nodes here then ?

024 You

Sep 24, 2026 06:10 AM EDT

`rt_zed_cam` is working. now i should create ros nodes to first figure out live pose error from gt node topics and then control the bot. You can use `ros2_ws/src/mpose_test` pkg as reference. how should we implement the ros nodes here then ?

025 Assistant • progress update

Sep 24, 2026 06:10 AM EDT

The environment check shows one integration issue: `mpose` uses Python 3.9, while the installed ROS 2 Humble bindings are built for Python 3.10. I'm checking the GT node's timestamp handling and pose frames so the design can preserve your working inference environment and produce meaningful live errors.

026 Assistant

Sep 24, 2026 06:12 AM EDT

I'd build three nodes for live evaluation first, then add a separate controller. Keep the working OpenCV/MegaPose process in `mpose`, with a small bridge into ROS.

I inspected your reference package. Two findings shape the implementation:

- `gt_pose_node.py` (`/home/abhi/dev/ros2_ws/src/mpose_test/mpose_test/gt_pose_node.py:19`) already buffers Vicon samples and interpolates position and orientation at image timestamps. We should reuse that logic. It currently **writes files rather than publishing a GT pose topic**.
- `mpose` uses Python 3.9; your installed ROS is **Humble with Python 3.10**. I confirmed that importing `rclpy` fails inside `mpose`. ROS binary bindings require a matching interpreter. [ROS documentation](#)

mermaid

```
graph TD
    A["rt_zed_cam.py - mpose environment"] --> B["mpose_bridge_node - ROS environment"]
    B --> C["Camera and marker Vicon topics"]
    C --> D["GT pose at matching timestamp"]
    D --> E["Live errors and CSV"]
    E --> F["turtlebot_controller_node"]
    F --> G["cmd_vel"]
```

1. Add a small output interface to `rt_zed_cam.py`.

After each processed frame, send a small JSON packet over localhost UDP using Python’s standard socket library. No additional Python dependency is needed.

The packet should contain:

text

```
{
  session_id:
  frame_index:
  image_time_ns:      # Existing host timestamp taken after cap.read()
  result_time_ns:
  status:             # TRACKING / NO_TARGET / NO_POSE
  T_camera_object:    # mpose.poses; absent when no valid pose
}
```

Use the **full transformation matrix** from `mpose.poses`, rather than reconstructing orientation from the displayed Euler angles. Publish invalid status explicitly when tracking is lost.

For the subsequent control stage, capture continuously into a single latest-frame slot. Inference should take the latest frame and its timestamp together, preventing a queue of old images.

2. `mpose_bridge_node`: **publish ROS messages**.

Run this node using ROS’s system Python. It receives the packets and publishes:

Topic	Type	Purpose
<code>/megapose/object_pose</code>	<code>geometry_msgs/PoseStamped</code>	Object pose in the left camera optical frame
<code>/megapose/status</code>	<code>diagnostic_msgs/DiagnosticArray</code>	Tracking state, frame index, inference latency and stream health

For the pose:

- `header.frame_id` = `"zed_left_camera_optical_frame"`
- `header.stamp` = `image_time_ns`, preserved from the source frame
- Position in metres; orientation as a normalized quaternion

A stream timeout should report stale data. The bridge must never refresh an old pose’s timestamp.

3. `gt_relative_pose_node`: **produce comparable ground truth**.

Reuse the subscriptions from your reference launch file

(/home/abhi/dev/ros2_ws/src/mpose_test/launch/record.launch.py):

text

```
/vicon/MARKER_YFORWARD/MARKER_YFORWARD
/vicon/ZED_CAM/ZED_CAM
```

Both are TransformStamped; keep their names configurable.

For each incoming MegaPose timestamp:

1. Find bracketing Vicon samples for the camera and marker.
2. Interpolate translation linearly and orientation using SLERP.
3. Convert the result into the **same object frame and camera frame** as MegaPose.
4. Publish /ground_truth/object_pose as PoseStamped, using exactly that timestamp.

The required transform is:

$$\begin{aligned} & \backslash \\ & T_{\{C,O\}}^{\{GT\}} \\ & = \\ & \left(T_{\{W,V_c\}} \backslash T_{\{V_c,C\}} \right)^{-1} \\ & T_{\{W,V_o\}} \backslash T_{\{V_o,O\}} \\ & \backslash \end{aligned}$$

Here, $\backslash(V_c)$ and $\backslash(V_o)$ are the Vicon camera and marker rigid-body frames; $\backslash(C)$ is the left optical frame; $\backslash(O)$ is the mesh frame. $\backslash(T_{\{A,B\}})$ maps coordinates from B into A.

The fixed mounting transforms matter. Vicon's marker origin may differ from the mesh origin, and the camera's tracked origin may differ from the left lens. Your existing frame conversion (/home/abhi/dev/mpose_rt/metrics_postprocess.py:155) applies an axis rotation with zero translation; we must verify those assumptions before treating the difference as estimator error.

Keep bounded Vicon history long enough to cover inference latency, and reject requests outside valid coverage.

4. pose_error_node: compare matched poses and report errors.

Match estimate and GT by their identical image timestamps. Compute:

- Signed translation error: $\backslash(\Delta p = p_{\{\text{estimate}\}} - p_{\{\text{GT}\}})$
- Position error: $\backslash(\backslash\Delta p)$, displayed in centimetres
- Orientation error: the angle of $\backslash(R_{\{\text{GT}\}}^{\text{top}} R_{\{\text{estimate}\}})$, displayed in degrees
- Rolling mean/RMSE, inference latency and unmatched-frame count

Publish timestamped diagnostics and save a CSV containing both poses and the calculated errors.

Timing needs explicit validation: OpenCV's host read timestamp is an approximation to exposure time. Camera buffering and differences between machine clocks can appear as pose error during motion. Start with a stationary marker, then test slow motion. Keep all timestamps on a common clock; don't compare an old estimate against the latest Vicon sample.

I'd place this in a new package:

text

```
ros2_ws/src/mpose_rt_ros/
├── mpose_rt_ros/
│   ├── mpose_bridge_node.py
│   ├── gt_relative_pose_node.py
│   └── pose_error_node.py
```

```
| └─ transforms.py
|   └─ config/experiment.yaml
|     └─ launch/evaluate.launch.py
```

Once evaluation works, add `turtlebot_controller_node.py`. It will transform the MegaPose marker pose into the robot's floor position and heading, then generate velocity commands toward a goal. Vicon remains an evaluation reference. Begin with rotation only, with explicit enable, speed limits and a stale-pose stop; verify the robot's actual `/cmd_vel` message type before connecting it.

The first implementation milestone should be the bridge plus synchronized GT/error publication, giving you live position/orientation error while the robot remains stationary.

027 You

Sep 24, 2026 06:19 AM EDT

So how does `rt-zed_cam` communicate with `mpose_bridge` node ? Is there no way to run megapose inside a ros node itself ? What are the simpler way of achieveing this communication ?

028 Assistant • progress update

Sep 24, 2026 06:19 AM EDT

I'll compare the communication options against your Python setup, including what it would take to run MegaPose directly inside a ROS node.

029 Assistant

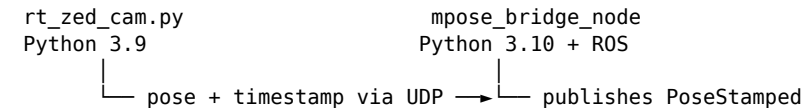
Sep 24, 2026 06:21 AM EDT

Yes, MegaPose can run directly inside a ROS node. I suggested a bridge because your working `mpose` environment uses Python 3.9, while your ROS 2 Humble bindings require Python 3.10. The bridge avoids changing that environment.

Currently, `rt_zed_cam.py` doesn't send anything to ROS; that communication still needs implementing.

In the proposed bridge approach, both programs run on the same computer:

text



After inference, `rt_zed_cam.py` sends a small packet containing the pose matrix, frame timestamp and tracking status to `127.0.0.1`. The bridge receives it, converts the rotation to a quaternion, and publishes a normal ROS message. **Only pose data crosses this connection; images stay in the inference process.** It doesn't require internet access.

There are three practical choices:

Approach	What changes	Main tradeoff
MegaPose directly in a ROS node	Set up a compatible Python 3.10 environment; wrap your existing runner	Simplest application structure, but environment setup needs validation
Small localhost UDP bridge	Add a sender to the current script and a small ROS receiver	Fewest dependency changes; we maintain the communication code
<code>rosbridge + roslibpy</code>	Run a standard ROS bridge server; publish from the current script through its Python client	Avoids writing a custom receiver, but adds packages and another service

For **direct ROS integration**, the node would own the existing objects:

text

```

MegaPoseNode
├── cv2.VideoCapture
├── ContourRunner
├── MegaPoseRunner
└── PoseStamped publisher

```

Its processing loop would simply capture an image, run your existing inference, and publish the result with that image's timestamp. We'd keep slow inference from blocking ROS callbacks by using a worker thread.

The environment requirement is real: sourcing ROS or adding its directories to PYTHONPATH cannot make a Python 3.10 compiled extension load in Python 3.9. The ROS documentation explicitly requires matching interpreters. [ROS Python environment guidance](#)

A separate environment based on **system Python 3.10**, with ROS access and MegaPose's dependencies installed, is the route I'd investigate. Your MegaPose environment files pin Python 3.9, but the package metadata doesn't explicitly prohibit 3.10. We still need to validate model loading, rendering and inference there.

For rosbridge, the connection would be:

text

```
rt_zed_cam.py + roslibpy → rosbridge server → /megapose/object_pose
```

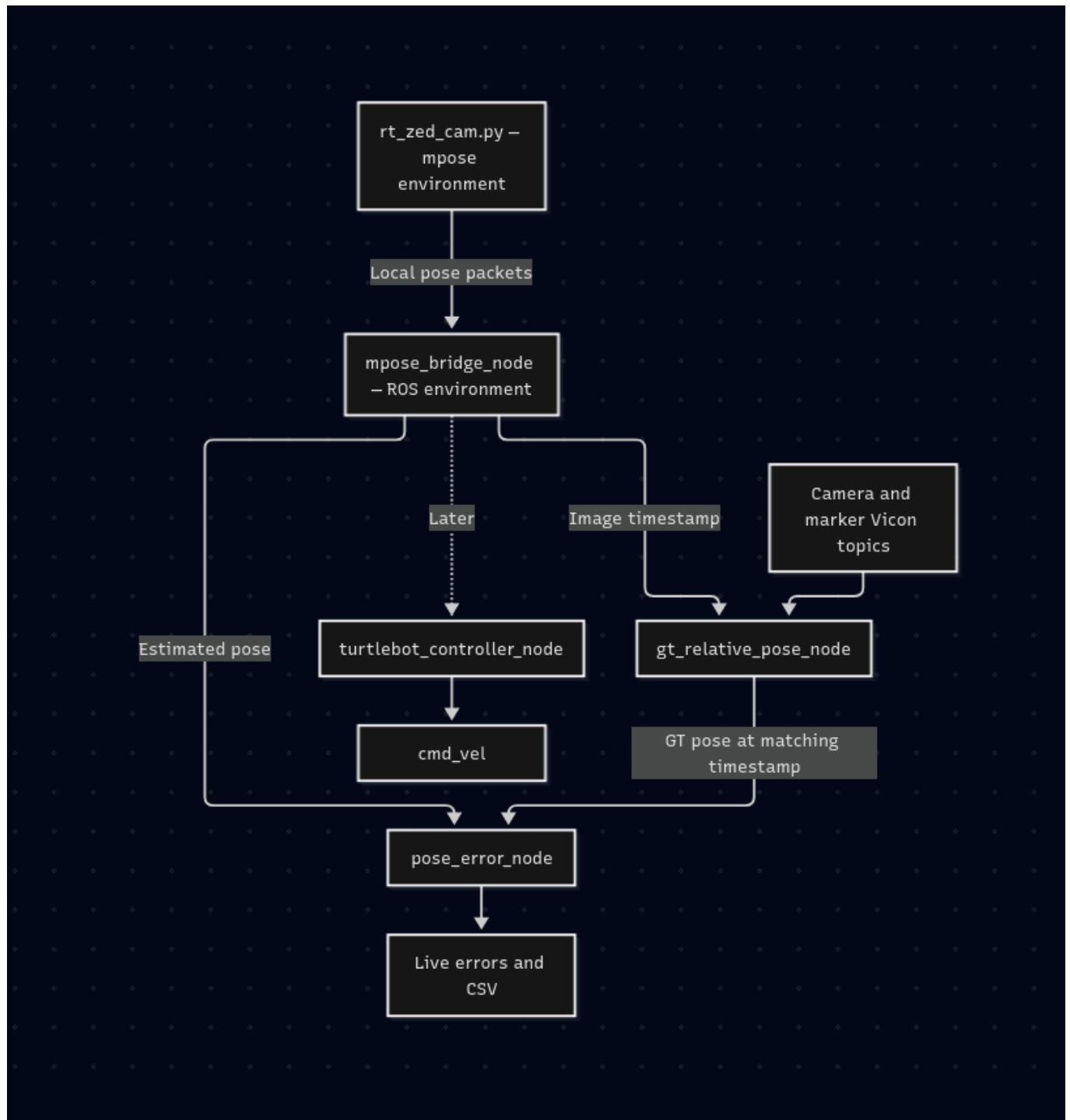
roslibpy communicates through WebSockets without importing rclpy, so the interpreters can differ. Its documentation still describes ROS 2 support as incomplete, so we'd verify basic topic publishing against Humble before adopting it. [roslibpy documentation](#)

My recommendation is to target a direct MegaPoseNode, starting by testing a separate Python 3.10 environment. If dependency migration becomes substantial, the small localhost bridge gives us a straightforward way to proceed with the already-working inference pipeline.

030 You

Sep 24, 2026 06:25 AM EDT

Okay let's go with 2nd approach. Modify the rt_mpose.py for inferece, give me the full file here. You can later combine gt_relative_pose_node with pose-error_node into a single node..



Uploaded flowchart

031 Assistant • progress update

Sep 24, 2026 06:25 AM EDT

I'll check `rt_mpose.py` and add the UDP pose sender while keeping inference in the `mpose` environment. I'll provide the full file here; the later GT comparison and error calculation can share one ROS node.

032 Assistant • progress update

Sep 24, 2026 06:26 AM EDT

I found `rt_mpose.py` in `ros2_ws/src/mpose_live/mpose_live`. I'm adding an explicit project path so it can import your existing inference code from there. The UDP packets will carry the original frame timestamp and full pose matrix, with a null pose on target loss so the bridge can detect it.

033 Assistant • progress update

Sep 24, 2026 06:29 AM EDT

The sender code passed syntax and focused checks for packet encoding, timestamps, target loss, shutdown status, and simulated UDP send failures. The replacement below uses 127.0.0.1:5005; the future bridge will need to preserve `image_time_ns` when publishing the pose. Live camera and UDP delivery remain untested here.

034 Assistant

Sep 24, 2026 06:31 AM EDT

Use this as the full replacement for `rt_mpose.py` (/home/abhi/dev/ros2_ws/src/mpose_live/mpose_live/rt_mpose.py). I couldn't save it because this session is read-only.

It preserves OpenCV inference and adds UDP pose packets to 127.0.0.1:5005. Missing poses are explicitly reported with `valid: false`. Syntax and focused logic checks passed; live camera and UDP delivery remain untested here.

python

```
#!/usr/bin/env python3
"""OpenCV ZED inference with JSON pose packets sent over localhost UDP.

Run with the mpose Python environment. No ROS or ZED SDK imports.
Images are raw left-camera frames; no undistortion is performed.
"""

import csv
import json
import socket
import sys
from argparse import ArgumentParser
from datetime import datetime
from pathlib import Path
from time import perf_counter, time_ns
from uuid import uuid4

import cv2
import numpy as np

def parse_args():
    parser = ArgumentParser(description=__doc__)
    parser.add_argument(
        "--project-root",
        type=Path,
        default=Path.home() / "dev/mpose_rt",
    )
    parser.add_argument("--cam_source", default="0")
    parser.add_argument("--mesh", required=True)
    parser.add_argument("--cam_file", default="zed_1080p_raw_calib.json")
    parser.add_argument("--model", default="megapose-1.0-RGB")
    parser.add_argument("--fps", type=int, default=15)
    parser.add_argument("--megapose-batch-size", type=int, default=16)
    parser.add_argument("--iou-threshold", type=float, default=0.5)
    parser.add_argument("--udp-port", type=int, default=5005)
    parser.add_argument("--frame-id", default="zed_left_camera_optical_frame")

    args = parser.parse_args()

    if args.fps <= 0 or args.megapose_batch_size <= 0:
        parser.error("FPS and batch size must be positive.")
    if not 0 <= args.iou_threshold <= 1:
        parser.error("IoU threshold must be between 0 and 1.")
    if not 1 <= args.udp_port <= 65535:
        parser.error("UDP port must be between 1 and 65535.")

    args.project_root = args.project_root.expanduser().resolve()
    if not (args.project_root / "mpose_runner.py").is_file():
        parser.error("--project-root must contain mpose_runner.py.")
```

```

return args

def resolve_input(value, root, folder):
    path = Path(value).expanduser()
    for candidate in (path, root / path, root / folder / path):
        if candidate.is_file():
            return candidate.resolve()
    raise FileNotFoundError(f"Cannot locate {value!r} in {root / folder}")

class PoseSender:
    def __init__(self, port, frame_id):
        self.destination = ("127.0.0.1", port)
        self.frame_id = frame_id
        self.session_id = uuid4().hex
        self.packet_seq = 0
        self.failed_sends = 0
        self.last_warning = -float("inf")

        self.sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        self.sock.setblocking(False)

    def send(
        self,
        status,
        frame_index=None,
        image_time_ns=None,
        result_time_ns=None,
        transform=None,
        inference_seconds=None,
    ):
        valid = status == "TRACKING"
        matrix = None

        if valid:
            matrix = np.asarray(transform, dtype=float)
            if (
                matrix.shape != (4, 4)
                or not np.isfinite(matrix).all()
                or not np.allclose(matrix[3], [0, 0, 0, 1])
                or matrix[2, 3] <= 0
            ):
                raise ValueError("Invalid current-frame object-to-camera pose.")

            if frame_index is None or image_time_ns is None:
                raise ValueError(
                    "A valid pose requires its source frame timestamp."
                )

            matrix = matrix.tolist()

        packet = {
            "schema_version": 1,
            "session_id": self.session_id,
            "packet_seq": self.packet_seq,
            "frame_index": frame_index,
            "image_time_ns": image_time_ns,
            "result_time_ns": (
                result_time_ns if result_time_ns is not None else time_ns()
            ),
            "timestamp_source": "host_read",
            "frame_id": self.frame_id,
            "object_id": "fiducial",
            "status": status,
            "valid": valid,
            "T_camera_object": matrix,
            "inference_seconds": inference_seconds,
        }

        payload = json.dumps(
            packet,
            allow_nan=False,
            separators=(",", ":"),
        ).encode("utf-8")

```

```

        self.packet_seq += 1

    try:
        self.sock.sendto(payload, self.destination)
        return True
    except OSError as error:
        self.failed_sends += 1
        now = perf_counter()
        if now - self.last_warning >= 2.0:
            print(f"UDP send failed: {error}", file=sys.stderr)
            self.last_warning = now
        return False

def close(self):
    self.sock.close()

def calculate_iou(a, b):
    if a is None or b is None:
        return 0.0

    a = np.asarray(a, dtype=float)
    b = np.asarray(b, dtype=float)

    if a.shape != (4,) or b.shape != (4,):
        return 0.0
    if not np.isfinite(a).all() or not np.isfinite(b).all():
        return 0.0

    intersection = np.maximum(
        0.0,
        np.minimum(a[2:], b[2:]) - np.maximum(a[:2], b[:2]),
    )
    overlap = float(np.prod(intersection))
    area_a = float(np.prod(np.maximum(0.0, a[2:] - a[:2])))
    area_b = float(np.prod(np.maximum(0.0, b[2:] - b[:2])))
    union = area_a + area_b - overlap

    return overlap / union if union > 0 else 0.0

def extract_left(frame, size):
    width, height = size
    expected = (height, width * 2, 3)

    if frame is None or frame.shape != expected or frame.dtype != np.uint8:
        actual = None if frame is None else frame.shape
        raise RuntimeError(
            f"Expected ZED stereo frame {expected}; got {actual}."
        )

    return frame[:, :width].copy()

def process_frame(mpose, detector, left_bgr, threshold):
    rgb = cv2.cvtColor(left_bgr, cv2.COLOR_BGR2RGB)
    _, boxes = detector.estimate(rgb)

    if boxes is None or len(boxes) == 0:
        mpose.reset_tracking()
        return left_bgr.copy(), None, "NO_TARGET"

    bbox = boxes[0]

    if mpose.tracking_active:
        try:
            predicted = mpose.mpose_bboxes()
        except ValueError:
            predicted = []

        if calculate_iou(bbox, predicted) < threshold:
            mpose.reset_tracking()

    if not mpose.tracking_active:
        mpose.load_detection(bbox)

```

```

pose = mpose.estimate(rgb)

if pose is None:
    mpose.reset_tracking()
    display, status = left_bgr.copy(), "NO_POSE"
else:
    display, status = mpose.draw_triaxis(), "TRACKING"

x1, y1, x2, y2 = map(int, bbox)
cv2.rectangle(display, (x1, y1), (x2, y2), (0, 255, 0), 2)

return display, pose, status

def main():
    args = parse_args()
    root = args.project_root

    # This script can live in ros2_ws while reusing the inference project.
    sys.path.insert(0, str(root))

    import torch
    from configs.config import COLOR_RANGES, OUT_RES
    from contour_runner import ContourRunner
    from mpose_runner import MegaPoseRunner

    mesh_path = resolve_input(args.mesh, root, "models")
    camera_path = resolve_input(args.cam_file, root, "configs")
    size = tuple(json.loads(camera_path.read_text())["img_size"])

    if size != tuple(OUT_RES):
        raise ValueError(
            f"Calibration {size} must match OUT_RES={OUT_RES}."
        )

    width, height = size

    sender = PoseSender(args.udp_port, args.frame_id)
    cap = video = pose_file = None
    window_created = False
    terminal_status = "STOPPED"
    frame_index = 0
    processed = 0
    started = perf_counter()

    try:
        sender.send("STARTING")

        mpose = MegaPoseRunner(
            mesh_path,
            "fiducial",
            args.model,
            camera_path,
            batch_size=args.megapose_batch_size,
        )
        detector = ContourRunner(COLOR_RANGES)

        source = (
            int(args.cam_source)
            if args.cam_source.isdecimal()
            else args.cam_source
        )
        cap = cv2.VideoCapture(source, cv2.CAP_V4L2)

        if not cap.isOpened():
            raise RuntimeError(
                f"Cannot open ZED device {args.cam_source}."
            )

        cap.set(cv2.CAP_PROP_FRAME_WIDTH, width * 2)
        cap.set(cv2.CAP_PROP_FRAME_HEIGHT, height)
        cap.set(cv2.CAP_PROP_FPS, args.fps)
        cap.set(cv2.CAP_PROP_BUFFERSIZE, 1) # Backend-dependent, best effort.

        output_dir = root / "outputs"

```

```

output_dir.mkdir(parents=True, exist_ok=True)
stem = "zed_pose_" + datetime.now().strftime("%Y%m%d_%H%M%S_%f")
video_path = output_dir / f"{stem}.mp4"
csv_path = output_dir / f"{stem}.csv"

video = cv2.VideoWriter(
    str(video_path),
    cv2.VideoWriter_fourcc(*"mp4v"),
    float(args.fps),
    size,
)
if not video.isOpened():
    raise RuntimeError(f"Cannot create {video_path}.")

pose_file = csv_path.open("w", newline="", encoding="utf-8")
rows = csv.writer(pose_file)
rows.writerow([
    "session_id",
    "frame_index",
    "image_time_ns",
    "result_time_ns",
    "status",
    "x_camera_m",
    "y_camera_m",
    "z_camera_m",
    "roll_camera_deg",
    "pitch_camera_deg",
    "yaw_camera_deg",
    "inference_seconds",
    "udp_send_ok",
])

cv2.namedWindow("ZED MegaPose", cv2.WINDOW_NORMAL)
window_created = True
cv2.resizeWindow("ZED MegaPose", 1280, 720)

print(f"UDP destination: 127.0.0.1:{args.udp_port}")
print(f"Session: {sender.session_id}")
print(f"Video: {video_path}\nPoses: {csv_path}")
print("q/Escape: quit; r: reset tracking.")
print("Timestamps are host read times, not hardware exposure times.")
print(f"Recorded MP4 playback uses fixed {args.fps} FPS.")

previous_time = perf_counter()

while True:
    ok, frame = cap.read()
    image_time_ns = time_ns()

    if not ok:
        raise RuntimeError("ZED stopped returning frames.")

    left_bgr = extract_left(frame, size)
    inference_start = perf_counter()

    try:
        display, pose, status = process_frame(
            mpose,
            detector,
            left_bgr,
            args.iou_threshold,
        )
    except torch.cuda.OutOfMemoryError:
        mpose.reset_tracking()
        torch.cuda.empty_cache()
        raise RuntimeError(
            "CUDA out of memory; retry --megapose-batch-size 4."
        ) from None

    inference_seconds = perf_counter() - inference_start
    result_time_ns = time_ns()

    # Never reuse a previous pose when this frame has no estimate.
    transform = mpose.poses if pose is not None else None
    sent = sender.send(

```

```

        status,
        frame_index,
        image_time_ns,
        result_time_ns,
        transform,
        inference_seconds,
    )

    now = perf_counter()
    loop_fps = 1.0 / max(now - previous_time, 1e-6)
    previous_time = now

    lines = [f"{status} | processed FPS: {loop_fps:.1f}"]
    if pose is not None:
        lines.append(
            f"x={pose[0]:.3f} y={pose[1]:.3f} "
            f"z={pose[2]:.3f} m"
        )

    for index, line in enumerate(lines):
        cv2.putText(
            display,
            line,
            (20, 35 + 30 * index),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.8,
            (0, 255, 0),
            2,
        )

    values = pose if pose is not None else [float("nan")] * 6
    rows.writerow([
        sender.session_id,
        frame_index,
        image_time_ns,
        result_time_ns,
        status,
        *values,
        f"{inference_seconds:.6f}",
        int(sent),
    ])
    pose_file.flush()

    video.write(display)
    cv2.imshow("ZED MegaPose", display)
    processed += 1
    frame_index += 1

    key = cv2.waitKey(1) & 0xFF

    if key in (ord("q"), 27):
        break
    if key == ord("r"):
        mpose.reset_tracking()
        sender.send("RESET")
    if cv2.getWindowProperty(
        "ZED MegaPose", cv2.WND_PROP_VISIBLE
    ) < 1:
        break

except KeyboardInterrupt:
    print("\nStopped.")
except Exception:
    terminal_status = "ERROR"
    raise
finally:
    # Best effort: the receiver must also enforce its own timeout.
    try:
        sender.send(terminal_status)
    finally:
        sender.close()
        if cap is not None:
            cap.release()
        if video is not None:
            video.release()

```

```

        if pose_file is not None:
            pose_file.close()
        if window_created:
            cv2.destroyAllWindows()

    print(
        f"Processed {processed} frames "
        f"in {perf_counter() - started:.1f}s."
    )
    print(f"Local UDP send failures: {sender.failed_sends}")

if __name__ == "__main__":
    main()

```

Run it using the mpose environment:

bash

```

conda activate mpose

python /home/abhi/dev/ros2_ws/src/mpose_live/mpose_live/rt_mpose.py \
--project-root /home/abhi/dev/mpose_rt \
--cam_source 0 \
--mesh BaselinePointer.obj \
--cam_file zed_1080p_raw_calib.json \
--udp-port 5005

```

The script runs even before the bridge starts; `udp_send_ok` means the local socket accepted the packet, not that a receiver received it. The future bridge should use `image_time_ns` **as the ROS pose timestamp** and reject stale data.

Later, we can combine GT interpolation and error calculation into one `gt_error_node`.

035 You

Sep 24, 2026 06:35 AM EDT

why do we need csv ?

036 Assistant

Sep 24, 2026 06:35 AM EDT

We don't need CSV for live operation. It was retained from your existing script to save poses for debugging and later comparison.

The live path is:

MegaPose → UDP → ROS bridge → GT comparison / robot control

We can remove CSV writing from `rt_mpose.py`. If you want recorded error measurements later, the combined `gt_error_node` can save the matched estimates, ground truth, and errors in one place.

037 You

Sep 24, 2026 06:40 AM EDT

So I can run this file outside `ros2_ws` and nodes inside this ws will pick this data being transmitted ?

038 Assistant

Sep 24, 2026 06:40 AM EDT

Yes. `rt_mpose.py` can run from any directory in the `mpose` environment. Its location relative to `ros2_ws` doesn't matter.

text

```

rt_mpose.py (mpose environment)
  | UDP → 127.0.0.1:5005
  ▼
mpose_bridge_node (ROS environment)
  | /megapose/object_pose
  ▼
GT/error and controller nodes

```

The **bridge node must be running and listening on port 5005**. Other ROS nodes subscribe to the topic it publishes; they don't receive the UDP packets automatically.

Both processes must run on the same machine with the current 127.0.0.1 configuration. Keep `--project-root /home/abhi/dev/mpose_rt` pointing to your inference code and assets.

The bridge still needs implementing.

039 You

Sep 24, 2026 06:45 AM EDT

how to check data stream on udp address ?

040 Assistant

Sep 24, 2026 06:45 AM EDT

Open another terminal and run this listener:

bash

```

python3 - <<'PY'
import json
import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind(("127.0.0.1", 5005))

print("Listening on 127.0.0.1:5005 – Ctrl+C to stop.", flush=True)

try:
    while True:
        data, sender = sock.recvfrom(65535)
        try:
            packet = json.loads(data)
            print(json.dumps(packet, indent=2), flush=True)
        except (ValueError, UnicodeDecodeError):
            print(f"Invalid JSON from {sender}: {data!r}")
except KeyboardInterrupt:
    pass
finally:
    sock.close()
PY

```

Then run `rt_mpose.py` in your `mpose` environment with `--udp-port 5005`.

You should see packets containing:

- status: TRACKING, NO_TARGET, etc.
- image_time_ns: frame timestamp.
- valid: whether this packet contains a pose.
- T_camera_object: the 4×4 pose matrix, or null.

If you get **“Address already in use”**, stop the bridge or another listener using port 5005 first. This listener temporarily takes the bridge's place.

041 You

Sep 24, 2026 06:49 AM EDT

Okay now write `ros_nodes` that consume `mpose` socket data.

042 Assistant • progress update

Sep 24, 2026 06:49 AM EDT

I'll inspect the ROS package and UDP sender, then prepare the receiver node that publishes poses and stream status. I'll also check what's available for the combined Vicon ground-truth and error node.

043 Assistant • clarification

Sep 24, 2026 06:49 AM EDT

For the Vicon error node, have you measured the fixed transforms from the tracked ZED body to its left optical frame, and from the tracked marker body to the MegaPose mesh frame?

- Not yet—make both configurable and leave evaluation disabled until set
- Yes—I can provide both transforms
- Use the existing `metrics_postprocess.py` assumptions for a preliminary comparison

044 Assistant • progress update

Sep 24, 2026 06:51 AM EDT

The sender is now `rt_live.py`, and its packets match the agreed format. I'll provide two ROS executables: a UDP-to-pose bridge and a combined GT/error node. They'll check timestamps, reject malformed or out-of-order data, and avoid CSV logging. This session is read-only, so I'll provide the files and check them in memory.

045 You

Sep 24, 2026 06:53 AM EDT

Reply to: For the Vicon error node, have you measured the fixed transforms from the tracked ZED body to its left optical frame, and from the tracked marker body to the MegaPose mesh frame?

Use the existing `metrics_postprocess.py` assumptions for a preliminary comparison

046 Assistant • progress update

Sep 24, 2026 06:54 AM EDT

I'll use the existing `metrics_postprocess.py` assumptions: its camera-axis rotation with zero translation offset, and coincident Vicon-marker and mesh frames. The GT/error node will label its results as preliminary and expose both transforms as parameters.

047 Assistant • progress update

Sep 24, 2026 06:57 AM EDT

The focused checks passed: packet validation, stream timeouts, stale-pose rejection, Vicon interpolation, and synthetic cases with zero error and a known 3 cm / 10° error. The ROS launch couldn't be exercised because it needs to create log files, which this session's read-only filesystem prevents.

048 Assistant

Sep 24, 2026 07:00 AM EDT

This adds **two ROS nodes**:

- mpose_bridge_node: receives UDP packets and publishes poses/status.
- gt_error_node: combines Vicon interpolation and pose-error calculation, using your existing frame assumptions.

No CSV logging. The files below must be copied into /home/abhi/dev/ros2_ws/src/mpose_live; this session is read-only.

1. Create mpose_live/ros_nodes.py:

python

```
#!/usr/bin/env python3
"""ROS 2 Humble UDP pose bridge and synchronized Vicon error evaluator."""

import json
import math
import socket
import time
from bisect import bisect_left
from collections import deque

import numpy as np
import rclpy
from diagnostic_msgs.msg import DiagnosticArray, DiagnosticStatus, KeyValue
from geometry_msgs.msg import PoseStamped, TransformStamped, Vector3Stamped
from rclpy.node import Node
from rclpy.qos import QoSProfile, ReliabilityPolicy
from scipy.spatial.transform import Rotation, Slerp

# Same camera-axis conversion as metrics_postprocess.py.
# T_A_B maps coordinates from frame B into frame A.
CAMERA_AXES = np.array([
    [-1., 0., 0., 0.],
    [0., 0., -1., 0.],
    [0., -1., 0., 0.],
    [0., 0., 0., 1.],
])

STATUSES = {
    "STARTING", "TRACKING", "NO_TARGET", "NO_POSE",
    "RESET", "ERROR", "STOPPED",
}

def stamp_ns(stamp):
    return stamp.sec * 1_000_000_000 + stamp.nanosec

def set_stamp(header, ns, frame):
    header.stamp.sec, header.stamp.nanosec = divmod(
        int(ns), 1_000_000_000
    )
    header.frame_id = frame

def checked_transform(value):
    matrix = np.asarray(value, dtype=float)

    if matrix.shape != (4, 4) or not np.isfinite(matrix).all():
        raise ValueError("Expected a finite 4x4 transform")

    rot = matrix[:3, :3]
    if (
        not np.allclose(matrix[3], [0, 0, 0, 1], atol=1e-6)
        or not np.allclose(rot.T @ rot, np.eye(3), atol=1e-3)
        or not np.isclose(np.linalg.det(rot), 1., atol=1e-3)
    )
```

```

):
    raise ValueError("Invalid rigid transform")

return matrix

def from_pose(position, orientation):
    p = np.array([position.x, position.y, position.z])
    q = np.array([
        orientation.x, orientation.y,
        orientation.z, orientation.w,
    ])

    if (
        not np.isfinite(p).all()
        or not np.isfinite(q).all()
        or np.linalg.norm(q) < 1e-12
    ):
        raise ValueError("Invalid position or quaternion")

    matrix = np.eye(4)
    matrix[:3, :3] = Rotation.from_quat(q).as_matrix()
    matrix[:3, 3] = p
    return matrix

def pose_message(matrix, ns, frame):
    msg = PoseStamped()
    set_stamp(msg.header, ns, frame)

    p = matrix[:3, 3]
    q = Rotation.from_matrix(matrix[:3, :3]).as_quat()

    (
        msg.pose.position.x,
        msg.pose.position.y,
        msg.pose.position.z,
    ) = map(float, p)

    (
        msg.pose.orientation.x,
        msg.pose.orientation.y,
        msg.pose.orientation.z,
        msg.pose.orientation.w,
    ) = map(float, q)

    return msg

def diagnostic(pub, name, message, level, ns, **values):
    msg = DiagnosticArray()
    set_stamp(msg.header, ns, "")

    status = DiagnosticStatus()
    status.name = name
    status.message = message
    status.level = level
    status.values = [
        KeyValue(key=str(k), value=str(v))
        for k, v in values.items()
    ]

    msg.status = [status]
    pub.publish(msg)

def positive(node, name, default):
    value = float(node.declare_parameter(name, default).value)
    if not math.isfinite(value) or value <= 0:
        raise ValueError(f"{name} must be finite and positive")
    return value

class PacketGate:
    """Validate packets before updating ordering state."""

```

```

def __init__(self, frame, object_id):
    self.frame = frame
    self.object_id = object_id
    self.session = None
    self.seq = -1
    self.result_ns = 0
    self.image_ns = 0

def accept(self, data, now_ns, packet_age_ns):
    p = json.loads(data)

    if (
        not isinstance(p, dict)
        or type(p.get("schema_version")) is not int
    ):
        raise ValueError("Missing packet schema")

    if p["schema_version"] != 1:
        raise ValueError("Unsupported packet schema")

    if (
        p.get("frame_id") != self.frame
        or p.get("object_id") != self.object_id
        or p.get("timestamp_source") != "host_read"
    ):
        raise ValueError(
            "Unexpected frame, object, or timestamp source"
        )

    session = p.get("session_id")
    seq = p.get("packet_seq")
    result_ns = p.get("result_time_ns")
    state = p.get("status")

    if not isinstance(session, str) or not session:
        raise ValueError("Missing session_id")

    if (
        type(seq) is not int
        or seq < 0
        or type(result_ns) is not int
    ):
        raise ValueError("Invalid sequence or result timestamp")

    if not isinstance(state, str) or state not in STATUSES:
        raise ValueError("Unknown sender status")

    if (
        type(p.get("valid")) is not bool
        or p["valid"] != (state == "TRACKING")
    ):
        raise ValueError("Inconsistent validity flag")

    if (
        result_ns <= 0
        or result_ns > now_ns + 50_000_000
        or now_ns - result_ns > packet_age_ns
    ):
        raise ValueError("Old packet or incompatible clock")

    same_session = session == self.session
    if (
        (same_session and seq <= self.seq)
        or result_ns < self.result_ns
        or (not same_session and result_ns <= self.result_ns)
    ):
        raise ValueError("Duplicate or out-of-order packet")

    matrix = None

    if state in {"TRACKING", "NO_TARGET", "NO_POSE"}:
        image_ns = p.get("image_time_ns")
        index = p.get("frame_index")

        if (

```

```

        type(image_ns) is not int
        or image_ns <= 0
        or image_ns > result_ns
        or type(index) is not int
        or index < 0
    ):
        raise ValueError(
            "Invalid source frame timestamp or index"
        )

    if same_session and image_ns <= self.image_ns:
        raise ValueError("Non-increasing image timestamp")

    elif (
        p.get("image_time_ns") is not None
        or p.get("frame_index") is not None
    ):
        raise ValueError(
            "Non-frame status must not contain an image timestamp"
        )

    if p["valid"]:
        matrix = checked_transform(p.get("T_camera_object"))
        if matrix[2, 3] <= 0:
            raise ValueError("Object is behind camera")
    elif p.get("T_camera_object") is not None:
        raise ValueError("Invalid result contains a pose")

    self.session = session
    self.seq = seq
    self.result_ns = result_ns

    if not same_session:
        self.image_ns = 0

    if state in {"TRACKING", "NO_TARGET", "NO_POSE"}:
        self.image_ns = p["image_time_ns"]

    return p, matrix

class UDPBridgeNode(Node):
    def __init__(self):
        super().__init__("mpose_bridge_node")

        if self.get_parameter("use_sim_time").value:
            raise ValueError(
                "Host timestamp packets require use_sim_time=false"
            )

        frame = self.declare_parameter(
            "camera_frame", "zed_left_camera_optical_frame"
        ).value
        object_id = self.declare_parameter(
            "object_id", "fiducial"
        ).value
        port = self.declare_parameter("udp_port", 5005).value

        if type(port) is not int or not 1 <= port <= 65535:
            raise ValueError("Invalid UDP port")

        self.timeout = positive(self, "stream_timeout_s", 2.0)
        self.max_age_ns = int(
            positive(self, "max_pose_age_s", 5.0) * 1e9
        )
        self.gate = PacketGate(frame, object_id)

        self.pose_pub = self.create_publisher(
            PoseStamped, "/megapose/object_pose", 100
        )
        self.status_pub = self.create_publisher(
            DiagnosticArray, "/megapose/status", 10
        )

        self.sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

```

```

try:
    self.sock.bind(("127.0.0.1", port))
    self.sock.setblocking(False)
except Exception:
    self.sock.close()
    raise

self.last_receive = None
self.last_packet = None
self.state = "WAITING"
self.rejected = 0

self.create_timer(0.01, self.receive)
self.create_timer(0.2, self.watchdog)

self.get_logger().info(f"Listening on 127.0.0.1:{port}")

def receive(self):
    # Bound work so other callbacks can run.
    for _ in range(100):
        try:
            data, _ = self.sock.recvfrom(65535)
        except BlockingIOError:
            break

        try:
            p, matrix = self.gate.accept(
                data,
                time.time_ns(),
                int(self.timeout * 1e9),
            )
        except (
            ValueError, TypeError, OverflowError, RecursionError
        ) as error:
            self.rejected += 1
            if self.rejected % 100 == 1:
                self.get_logger().warning(
                    f"Rejected UDP packet: {error}"
                )
            continue

        self.last_receive = time.monotonic()
        self.last_packet = p
        self.state = p["status"]

        if matrix is not None:
            age_ns = time.time_ns() - p["image_time_ns"]
            if age_ns > self.max_age_ns:
                self.state = "STALE_POSE"
            else:
                self.pose_pub.publish(
                    pose_message(
                        matrix,
                        p["image_time_ns"],
                        self.gate.frame,
                    )
                )

        self.publish_status()

def watchdog(self):
    if self.last_receive is not None:
        if time.monotonic() - self.last_receive > self.timeout:
            self.state = "STREAM_TIMEOUT"
        elif (
            self.state == "TRACKING"
            and time.time_ns() - self.last_packet["image_time_ns"]
            > self.max_age_ns
        ):
            self.state = "STALE_POSE"

    self.publish_status()

def publish_status(self):
    packet = self.last_packet or {}

```

```

image_ns = packet.get("image_time_ns")
age = (
    (time.time_ns() - image_ns) / 1e9
    if image_ns else None
)

diagnostic(
    self.status_pub,
    "megapose_stream",
    self.state,
    (
        DiagnosticStatus.OK
        if self.state == "TRACKING"
        else DiagnosticStatus.WARN
    ),
    time.time_ns(),
    valid=self.state == "TRACKING",
    session_id=self.gate.session,
    packet_seq=self.gate.seq,
    image_time_ns=image_ns,
    pose_age_s=age,
    rejected_packets=self.rejected,
)

```

```

class PoseHistory:
    def __init__(self, duration_ns):
        self.duration_ns = duration_ns
        self.stamps = []
        self.matrices = []

    def add(self, ns, matrix):
        if self.stamps and ns <= self.stamps[-1]:
            return

        self.stamps.append(ns)
        self.matrices.append(matrix)

        cut = max(
            0,
            bisect_left(self.stamps, ns - self.duration_ns) - 1,
            len(self.stamps) - 10000,
        )
        if cut:
            del self.stamps[:cut]
            del self.matrices[:cut]

    def at(self, ns, max_gap_ns):
        i = bisect_left(self.stamps, ns)

        if i < len(self.stamps) and self.stamps[i] == ns:
            return self.matrices[i]

        if i == len(self.stamps):
            raise LookupError(
                "Waiting for Vicon samples after the image"
            )

        if i == 0:
            raise ValueError("Image is older than Vicon history")

        t0, t1 = self.stamps[i - 1:i + 1]
        if t1 - t0 > max_gap_ns:
            raise ValueError("Vicon interpolation gap is too large")

        a, b = self.matrices[i - 1:i + 1]
        alpha = (ns - t0) / (t1 - t0)

        matrix = np.eye(4)
        matrix[:3, 3] = (
            (1 - alpha) * a[:3, 3] + alpha * b[:3, 3]
        )
        matrix[:3, :3] = Slerp(
            [0., 1.],
            Rotation.from_matrix(

```

```

        np.stack([a[:3, :3], b[:3, :3]])
    ),
)(alpha).as_matrix()

return matrix

class GTErrorNode(Node):
    def __init__(self):
        super().__init__("gt_error_node")

        if self.get_parameter("use_sim_time").value:
            raise ValueError(
                "Live host timestamps require use_sim_time=false"
            )

        self.frame = self.declare_parameter(
            "camera_frame", "zed_left_camera_optical_frame"
        ).value
        self.ready = self.declare_parameter(
            "evaluation_enabled", True
        ).value
        self.preliminary = self.declare_parameter(
            "preliminary", True
        ).value

        # Both parameters are row-major, flattened 4x4 transforms.
        self.T_vc_c = checked_transform(
            np.array(
                self.declare_parameter(
                    "camera_body_to_optical",
                    CAMERA_AXES.ravel().tolist(),
                ).value
            ).reshape(4, 4)
        )
        self.T_vo_o = checked_transform(
            np.array(
                self.declare_parameter(
                    "marker_body_to_mesh",
                    np.eye(4).ravel().tolist(),
                ).value
            ).reshape(4, 4)
        )

        history_ns = int(positive(self, "history_s", 20.0) * 1e9)
        self.gap_ns = int(
            positive(self, "max_vicon_gap_s", 0.05) * 1e9
        )
        self.wait_s = positive(self, "sync_wait_s", 1.0)

        self.history = {
            k: PoseHistory(history_ns)
            for k in ("camera", "marker")
        }
        self.frames = {}
        self.frame_fault = False
        self.pending = deque()
        self.last_estimate = None
        self.last_log = 0.
        self.matched = 0
        self.missed = 0

        self.gt_pub = self.create_publisher(
            PoseStamped, "/ground_truth/object_pose", 100
        )
        self.error_pub = self.create_publisher(
            DiagnosticArray, "/megapose/errors", 10
        )
        self.delta_pub = self.create_publisher(
            Vector3Stamped, "/megapose/translation_error", 10
        )

        qos = QoSProfile(
            depth=1000,
            reliability=ReliabilityPolicy.BEST_EFFORT,

```

```

    )

    for name, parameter, default in [
        (
            "camera",
            "camera_vicon_topic",
            "/vicon/ZED_CAM/ZED_CAM",
        ),
        (
            "marker",
            "marker_vicon_topic",
            "/vicon/MARKER_YFORWARD/MARKER_YFORWARD",
        ),
    ]:
        topic = self.declare_parameter(parameter, default).value
        self.create_subscription(
            TransformStamped,
            topic,
            lambda msg, key=name: self.on_vicon(key, msg),
            qos,
        )

    self.create_subscription(
        PoseStamped,
        "/megapose/object_pose",
        self.on_estimate,
        100,
    )
    self.create_timer(0.02, self.process)
    self.create_timer(1.0, self.idle_status)

    if self.preliminary:
        self.get_logger().warning(
            "Using metrics_postprocess.py frame assumptions; "
            "errors are preliminary."
        )

def report(self, state, ns=None, **values):
    diagnostic(
        self.error_pub,
        "megapose_vs_vicon",
        state,
        (
            DiagnosticStatus.OK
            if state == "MATCHED"
            else DiagnosticStatus.WARN
        ),
        time.time_ns() if ns is None else ns,
        matched=self.matched,
        missed=self.missed,
        preliminary=self.preliminary,
        **values,
    )

def idle_status(self):
    if not self.ready:
        self.report("EVALUATION_DISABLED")
    elif self.frame_fault:
        self.report("VICON_FRAME_MISMATCH")
    elif (
        self.last_estimate is None
        or time.monotonic() - self.last_estimate > 2.0
    ):
        self.report("WAITING_FOR_ESTIMATE")

def on_vicon(self, name, msg):
    ns = stamp_ns(msg.header.stamp)
    pair = (msg.header.frame_id, msg.child_frame_id)

    if ns <= 0 or not all(pair) or self.frame_fault:
        return

    if name in self.frames and self.frames[name] != pair:
        self.frame_fault = True
        self.report("VICON_FRAME_MISMATCH")

```

```

        return

    try:
        matrix = from_pose(
            msg.transform.translation,
            msg.transform.rotation,
        )
    except ValueError:
        return

    self.frames[name] = pair

    if (
        len(self.frames) == 2
        and self.frames["camera"][0] != self.frames["marker"][0]
    ):
        self.frame_fault = True
        self.report("VICON_FRAME_MISMATCH")
        return

    self.history[name].add(ns, matrix)

def on_estimate(self, msg):
    self.last_estimate = time.monotonic()

    if not self.ready or self.frame_fault:
        return

    ns = stamp_ns(msg.header.stamp)
    if ns <= 0 or msg.header.frame_id != self.frame:
        self.report("INVALID_ESTIMATE_FRAME")
        return

    try:
        matrix = from_pose(
            msg.pose.position,
            msg.pose.orientation,
        )
    except ValueError:
        self.report("INVALID_ESTIMATE")
        return

    if len(self.pending) >= 200:
        self.pending.popleft()
        self.missed += 1

    self.pending.append((ns, matrix, time.monotonic()))

def process(self):
    if self.frame_fault:
        self.pending.clear()
        return

    while self.pending:
        ns, estimate, received = self.pending[0]

        try:
            camera = self.history["camera"].at(ns, self.gap_ns)
            marker = self.history["marker"].at(ns, self.gap_ns)
        except LookupError as error:
            if time.monotonic() - received < self.wait_s:
                break

            self.pending.popleft()
            self.missed += 1
            self.report("VICON_TIMEOUT", ns, reason=error)
            continue
        except ValueError as error:
            self.pending.popleft()
            self.missed += 1
            self.report("UNMATCHED", ns, reason=error)
            continue

        self.pending.popleft()

```

```

# T_C_0 = inverse(T_W_Vc * T_Vc_C) * T_W_Vo * T_Vo_0
gt = (
    np.linalg.inv(camera @ self.T_vc_c)
    @ marker
    @ self.T_vo_o
)

delta = estimate[:3, 3] - gt[:3, 3]
position_error = float(np.linalg.norm(delta))
rotation_error = float(
    np.degrees(
        Rotation.from_matrix(
            gt[:3, :3].T @ estimate[:3, :3]
        ).magnitude()
    )
)

self.gt_pub.publish(
    pose_message(gt, ns, self.frame)
)

msg = Vector3Stamped()
set_stamp(msg.header, ns, self.frame)
msg.vector.x, msg.vector.y, msg.vector.z = map(
    float, delta
)
self.delta_pub.publish(msg)

self.matched += 1
self.report(
    "MATCHED",
    ns,
    position_error_m=position_error,
    rotation_error_deg=rotation_error,
    estimate_age_s=(time.time_ns() - ns) / 1e9,
)

if time.monotonic() - self.last_log >= 1.0:
    self.get_logger().info(
        f"Position error: {position_error * 100:.2f} cm; "
        f"rotation error: {rotation_error:.2f} deg"
    )
    self.last_log = time.monotonic()

```

```

def run(node_type, args=None):
    rclpy.init(args=args)
    node = None

    try:
        node = node_type()
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        if node is not None:
            if isinstance(node, UDPBridgeNode):
                node.sock.close()
            node.destroy_node()

        if rclpy.ok():
            rclpy.shutdown()

```

```

def bridge_main(args=None):
    run(UDPBridgeNode, args)

```

```

def gt_error_main(args=None):
    run(GTErrorNode, args)

```

2. Replace setup.py:

python

```

from glob import glob
from setuptools import find_packages, setup

package_name = "mpose_live"

setup(
    name=package_name,
    version="0.0.0",
    packages=find_packages(exclude=["test"]),
    data_files=[
        (
            "share/ament_index/resource_index/packages",
            ["resource/" + package_name],
        ),
        ("share/" + package_name, ["package.xml"]),
        (
            "share/" + package_name + "/launch",
            glob("launch/*.launch.py"),
        ),
    ],
    install_requires=["setuptools"],
    zip_safe=True,
    maintainer="abhi",
    maintainer_email="coderadonis@gmail.com",
    description="MegaPose UDP bridge and live Vicon pose comparison",
    license="TODO: License declaration",
    entry_points={
        "console_scripts": [
            "mpose_bridge_node = mpose_live.ros_nodes:bridge_main",
            "gt_error_node = mpose_live.ros_nodes:gt_error_main",
        ],
    },
)

```

3. Replace package.xml:

xml

```

<?xml version="1.0"?>
<package format="3">
  <name>mpose_live</name>
  <version>0.0.0</version>
  <description>MegaPose UDP bridge and live Vicon pose comparison</description>
  <maintainer email="coderadonis@gmail.com">abhi</maintainer>
  <license>TODO: License declaration</license>

  <buildtool_depend>ament_python</buildtool_depend>

  <exec_depend>rclpy</exec_depend>
  <exec_depend>geometry_msgs</exec_depend>
  <exec_depend>diagnostic_msgs</exec_depend>
  <exec_depend>python3-numpy</exec_depend>
  <exec_depend>python3-scipy</exec_depend>
  <exec_depend>launch</exec_depend>
  <exec_depend>launch_ros</exec_depend>

  <test_depend>ament_copyright</test_depend>
  <test_depend>ament_flake8</test_depend>
  <test_depend>ament_pep257</test_depend>
  <test_depend>python3-pytest</test_depend>

  <export>
    <build_type>ament_python</build_type>
  </export>
</package>

```

4. Replace launch/demo.launch.py:

python

```

from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():

```

```

return LaunchDescription([
    Node(
        package="mpose_live",
        executable="mpose_bridge_node",
        output="screen",
        parameters=[{
            "use_sim_time": False,
            "udp_port": 5005,
            "stream_timeout_s": 2.0,
            "max_pose_age_s": 5.0,
        }],
    ),
    Node(
        package="mpose_live",
        executable="gt_error_node",
        output="screen",
        parameters=[{
            "use_sim_time": False,
            "evaluation_enabled": True,
            "preliminary": True,
            "camera_vicon_topic": "/vicon/ZED_CAM/ZED_CAM",
            "marker_vicon_topic": (
                "/vicon/MARKER_YFORWARD/MARKER_YFORWARD"
            ),
            "history_s": 20.0,
            "max_vicon_gap_s": 0.05,
            "sync_wait_s": 1.0,
        }],
    ),
])

```

Keep the existing `setup.cfg`, `resource/mpose_live`, and `mpose_live/__init__.py`.

Build and launch from a ROS terminal, outside the `mpose` environment:

bash

```

conda deactivate
source /opt/ros/humble/setup.bash

cd /home/abhi/dev/ros2_ws
/usr/bin/python3 /usr/bin/colcon build \
  --symlink-install --packages-select mpose_live

source install/setup.bash
ros2 launch mpose_live demo.launch.py

```

Stop `check_udp.py` **first**, since the bridge needs exclusive use of port 5005. Run your working `rt_live.py` separately in `mpose`, and start the existing Vicon topic publishers.

Inspect the outputs:

bash

```

ros2 topic echo /megapose/object_pose
ros2 topic echo /megapose/status
ros2 topic echo /megapose/errors

```

The error node also publishes `/ground_truth/object_pose` and `/megapose/translation_error`. Both retain the estimate's original image timestamp.

Packet handling, timeout behavior, interpolation, and synthetic **3 cm / 10°** error cases passed focused checks. Live ROS launch was not tested because the read-only filesystem prevents ROS from creating its log files.

049 You

Sep 24, 2026 07:03 AM EDT

Separate the `ros_nodes.py` into two nodes.

050 Assistant • progress update

Sep 24, 2026 07:03 AM EDT

I'll split it into a UDP bridge file and a GT/error file, then update the ROS entry points. This workspace is read-only, so I'll provide the replacement files here.

051 Assistant • progress update

Sep 24, 2026 07:04 AM EDT

The bridge will handle UDP packets and pose publishing; the GT/error node will handle Vicon synchronization and comparison. Topic names and parameters stay the same. The package on disk still has the original scaffold, so the setup and launch configuration from the previous version will also be needed.

052 Assistant

Sep 24, 2026 07:07 AM EDT

Use two separate files:

- mpose_bridge_node.py: receives UDP packets and publishes estimated poses.
- gt_error_node.py: synchronizes Vicon data and publishes pose errors.

The topics, parameters, and preliminary calibration assumptions stay the same. The workspace is read-only, so save the following files manually.

Save as ros2_ws/src/mpose_live/mpose_live/mpose_bridge_node.py:

python

```
#!/usr/bin/env python3
"""Receive MegaPose UDP packets and publish ROS poses and stream status."""

import json
import math
import socket
import time

import numpy as np
import rclpy
from diagnostic_msgs.msg import DiagnosticArray, DiagnosticStatus, KeyValue
from geometry_msgs.msg import PoseStamped
from rclpy.node import Node
from scipy.spatial.transform import Rotation

STATUSES = {
    "STARTING",
    "TRACKING",
    "NO_TARGET",
    "NO_POSE",
    "RESET",
    "ERROR",
    "STOPPED",
}

def set_stamp(header, ns, frame):
    header.stamp.sec, header.stamp.nanosec = divmod(int(ns), 1_000_000_000)
    header.frame_id = frame

def checked_transform(value):
    matrix = np.asarray(value, dtype=float)

    if matrix.shape != (4, 4) or not np.isfinite(matrix).all():
        raise ValueError("Expected a finite 4x4 transform")
```

```

    rot = matrix[:3, :3]
    if (
        not np.allclose(matrix[3], [0, 0, 0, 1], atol=1e-6)
        or not np.allclose(rot.T @ rot, np.eye(3), atol=1e-3)
        or not np.isclose(np.linalg.det(rot), 1.0, atol=1e-3)
    ):
        raise ValueError("Invalid rigid transform")

    return matrix

def pose_message(matrix, ns, frame):
    msg = PoseStamped()
    set_stamp(msg.header, ns, frame)

    p = matrix[:3, 3]
    q = Rotation.from_matrix(matrix[:3, :3]).as_quat()

    msg.pose.position.x, msg.pose.position.y, msg.pose.position.z = map(float, p)
    (
        msg.pose.orientation.x,
        msg.pose.orientation.y,
        msg.pose.orientation.z,
        msg.pose.orientation.w,
    ) = map(float, q)

    return msg

def diagnostic(pub, name, message, level, ns, **values):
    msg = DiagnosticArray()
    set_stamp(msg.header, ns, "")

    status = DiagnosticStatus()
    status.name = name
    status.message = message
    status.level = level
    status.values = [
        KeyValue(key=str(k), value=str(v)) for k, v in values.items()
    ]

    msg.status = [status]
    pub.publish(msg)

def positive(node, name, default):
    value = float(node.declare_parameter(name, default).value)
    if not math.isfinite(value) or value <= 0:
        raise ValueError(f"{name} must be finite and positive")
    return value

class PacketGate:
    """Validate packets before updating ordering state."""

    def __init__(self, frame, object_id):
        self.frame = frame
        self.object_id = object_id
        self.session = None
        self.seq = -1
        self.result_ns = 0
        self.image_ns = 0

    def accept(self, data, now_ns, packet_age_ns):
        p = json.loads(data)

        if not isinstance(p, dict) or type(p.get("schema_version")) is not int:
            raise ValueError("Missing packet schema")

        if p["schema_version"] != 1:
            raise ValueError("Unsupported packet schema")

        if (
            p.get("frame_id") != self.frame
            or p.get("object_id") != self.object_id

```

```

        or p.get("timestamp_source") != "host_read"
    ):
        raise ValueError("Unexpected frame, object, or timestamp source")

    session = p.get("session_id")
    seq = p.get("packet_seq")
    result_ns = p.get("result_time_ns")
    state = p.get("status")

    if not isinstance(session, str) or not session:
        raise ValueError("Missing session_id")

    if type(seq) is not int or seq < 0 or type(result_ns) is not int:
        raise ValueError("Invalid sequence or result timestamp")

    if not isinstance(state, str) or state not in STATUSES:
        raise ValueError("Unknown sender status")

    if type(p.get("valid")) is not bool or p["valid"] != (state == "TRACKING"):
        raise ValueError("Inconsistent validity flag")

    if (
        result_ns <= 0
        or result_ns > now_ns + 50_000_000
        or now_ns - result_ns > packet_age_ns
    ):
        raise ValueError("Old packet or incompatible clock")

    same_session = session == self.session
    if (
        (same_session and seq <= self.seq)
        or result_ns < self.result_ns
        or (not same_session and result_ns <= self.result_ns)
    ):
        raise ValueError("Duplicate or out-of-order packet")

    matrix = None

    if state in {"TRACKING", "NO_TARGET", "NO_POSE"}:
        image_ns = p.get("image_time_ns")
        index = p.get("frame_index")

        if (
            type(image_ns) is not int
            or image_ns <= 0
            or image_ns > result_ns
            or type(index) is not int
            or index < 0
        ):
            raise ValueError("Invalid source frame timestamp or index")

        if same_session and image_ns <= self.image_ns:
            raise ValueError("Non-increasing image timestamp")

    elif (
        p.get("image_time_ns") is not None
        or p.get("frame_index") is not None
    ):
        raise ValueError("Non-frame status must not contain an image timestamp")

    if p["valid"]:
        matrix = checked_transform(p.get("T_camera_object"))
        if matrix[2, 3] <= 0:
            raise ValueError("Object is behind camera")
    elif p.get("T_camera_object") is not None:
        raise ValueError("Invalid result contains a pose")

    self.session = session
    self.seq = seq
    self.result_ns = result_ns

    if not same_session:
        self.image_ns = 0

    if state in {"TRACKING", "NO_TARGET", "NO_POSE"}:

```

```

        self.image_ns = p["image_time_ns"]

    return p, matrix

class UDPBridgeNode(Node):
    def __init__(self):
        super().__init__("mpose_bridge_node")

        if self.get_parameter("use_sim_time").value:
            raise ValueError("Host timestamp packets require use_sim_time=false")

        frame = self.declare_parameter(
            "camera_frame", "zed_left_camera_optical_frame"
        ).value
        object_id = self.declare_parameter("object_id", "fiducial").value
        port = self.declare_parameter("udp_port", 5005).value

        if type(port) is not int or not 1 <= port <= 65535:
            raise ValueError("Invalid UDP port")

        self.timeout = positive(self, "stream_timeout_s", 2.0)
        self.max_age_ns = int(positive(self, "max_pose_age_s", 5.0) * 1e9)
        self.gate = PacketGate(frame, object_id)

        self.pose_pub = self.create_publisher(
            PoseStamped, "/megapose/object_pose", 100
        )
        self.status_pub = self.create_publisher(
            DiagnosticArray, "/megapose/status", 10
        )

        self.sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        try:
            self.sock.bind(("127.0.0.1", port))
            self.sock.setblocking(False)
        except Exception:
            self.sock.close()
            raise

        self.last_receive = None
        self.last_packet = None
        self.state = "WAITING"
        self.rejected = 0

        self.create_timer(0.01, self.receive)
        self.create_timer(0.2, self.watchdog)

        self.get_logger().info(f"Listening on 127.0.0.1:{port}")

    def receive(self):
        # Bound work so other callbacks can run.
        for _ in range(100):
            try:
                data, _ = self.sock.recvfrom(65535)
            except BlockingIOError:
                break

            try:
                packet, matrix = self.gate.accept(
                    data, time.time_ns(), int(self.timeout * 1e9)
                )
            except (ValueError, TypeError, OverflowError, RecursionError) as error:
                self.rejected += 1
                if self.rejected % 100 == 1:
                    self.get_logger().warning(f"Rejected UDP packet: {error}")
                continue

            self.last_receive = time.monotonic()
            self.last_packet = packet
            self.state = packet["status"]

            if matrix is not None:
                if time.time_ns() - packet["image_time_ns"] > self.max_age_ns:
                    self.state = "STALE_POSE"

```

```

        else:
            self.pose_pub.publish(
                pose_message(
                    matrix,
                    packet["image_time_ns"],
                    self.gate.frame,
                )
            )

        self.publish_status()

def watchdog(self):
    if self.last_receive is not None:
        if time.monotonic() - self.last_receive > self.timeout:
            self.state = "STREAM_TIMEOUT"
        elif (
            self.state == "TRACKING"
            and time.time_ns() - self.last_packet["image_time_ns"]
            > self.max_age_ns
        ):
            self.state = "STALE_POSE"

    self.publish_status()

def publish_status(self):
    packet = self.last_packet or {}
    image_ns = packet.get("image_time_ns")
    age = (time.time_ns() - image_ns) / 1e9 if image_ns else None

    diagnostic(
        self.status_pub,
        "megapose_stream",
        self.state,
        DiagnosticStatus.OK
        if self.state == "TRACKING"
        else DiagnosticStatus.WARN,
        time.time_ns(),
        valid=self.state == "TRACKING",
        session_id=self.gate.session,
        packet_seq=self.gate.seq,
        image_time_ns=image_ns,
        pose_age_s=age,
        rejected_packets=self.rejected,
    )

def main(args=None):
    rclpy.init(args=args)
    node = None

    try:
        node = UDPBridgeNode()
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        if node is not None:
            node.sock.close()
            node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()

if __name__ == "__main__":
    main()

```

Save as `ros2_ws/src/mpose_live/mpose_live/gt_error_node.py`:

python

```

#!/usr/bin/env python3
"""Compare MegaPose with Vicon ground truth at the image timestamp."""

import math
import time

```

```

from bisect import bisect_left
from collections import deque

import numpy as np
import rclpy
from diagnostic_msgs.msg import DiagnosticArray, DiagnosticStatus, KeyValue
from geometry_msgs.msg import PoseStamped, TransformStamped, Vector3Stamped
from rclpy.node import Node
from rclpy.qos import QoSProfile, ReliabilityPolicy
from scipy.spatial.transform import Rotation, Slerp

# Existing metrics_postprocess.py assumptions:
# camera optical frame expressed in the tracked camera-body frame.
CAMERA_AXES = np.array([
    [-1.0, 0.0, 0.0, 0.0],
    [0.0, 0.0, -1.0, 0.0],
    [0.0, -1.0, 0.0, 0.0],
    [0.0, 0.0, 0.0, 1.0],
])

def stamp_ns(stamp):
    return stamp.sec * 1_000_000_000 + stamp.nanosec

def set_stamp(header, ns, frame):
    header.stamp.sec, header.stamp.nanosec = divmod(int(ns), 1_000_000_000)
    header.frame_id = frame

def checked_transform(value):
    matrix = np.asarray(value, dtype=float)

    if matrix.shape != (4, 4) or not np.isfinite(matrix).all():
        raise ValueError("Expected a finite 4x4 transform")

    rot = matrix[:3, :3]
    if (
        not np.allclose(matrix[3], [0, 0, 0, 1], atol=1e-6)
        or not np.allclose(rot.T @ rot, np.eye(3), atol=1e-3)
        or not np.isclose(np.linalg.det(rot), 1.0, atol=1e-3)
    ):
        raise ValueError("Invalid rigid transform")

    return matrix

def from_pose(position, orientation):
    p = np.array([position.x, position.y, position.z])
    q = np.array([
        orientation.x,
        orientation.y,
        orientation.z,
        orientation.w,
    ])

    if (
        not np.isfinite(p).all()
        or not np.isfinite(q).all()
        or np.linalg.norm(q) < 1e-12
    ):
        raise ValueError("Invalid position or quaternion")

    matrix = np.eye(4)
    matrix[:3, :3] = Rotation.from_quat(q).as_matrix()
    matrix[:3, 3] = p
    return matrix

def pose_message(matrix, ns, frame):
    msg = PoseStamped()
    set_stamp(msg.header, ns, frame)

    p = matrix[:3, 3]

```

```

q = Rotation.from_matrix(matrix[:3, :3]).as_quat()

msg.pose.position.x, msg.pose.position.y, msg.pose.position.z = map(float, p)
(
    msg.pose.orientation.x,
    msg.pose.orientation.y,
    msg.pose.orientation.z,
    msg.pose.orientation.w,
) = map(float, q)

return msg

def diagnostic(pub, name, message, level, ns, **values):
    msg = DiagnosticArray()
    set_stamp(msg.header, ns, "")

    status = DiagnosticStatus()
    status.name = name
    status.message = message
    status.level = level
    status.values = [
        KeyValue(key=str(k), value=str(v)) for k, v in values.items()
    ]

    msg.status = [status]
    pub.publish(msg)

def positive(node, name, default):
    value = float(node.declare_parameter(name, default).value)
    if not math.isfinite(value) or value <= 0:
        raise ValueError(f"{name} must be finite and positive")
    return value

class PoseHistory:
    """Keep timestamped transforms and interpolate within their coverage."""

    def __init__(self, duration_ns):
        self.duration_ns = duration_ns
        self.stamps = []
        self.matrices = []

    def add(self, ns, matrix):
        if self.stamps and ns <= self.stamps[-1]:
            return

        self.stamps.append(ns)
        self.matrices.append(matrix)

        cut = max(
            0,
            bisect_left(self.stamps, ns - self.duration_ns) - 1,
            len(self.stamps) - 10000,
        )

        if cut:
            del self.stamps[:cut]
            del self.matrices[:cut]

    def at(self, ns, max_gap_ns):
        i = bisect_left(self.stamps, ns)

        if i < len(self.stamps) and self.stamps[i] == ns:
            return self.matrices[i]

        if i == len(self.stamps):
            raise LookupError("Waiting for Vicon samples after the image")

        if i == 0:
            raise ValueError("Image is older than Vicon history")

        t0, t1 = self.stamps[i - 1:i + 1]
        if t1 - t0 > max_gap_ns:

```

```

        raise ValueError("Vicon interpolation gap is too large")

    a, b = self.matrices[i - 1:i + 1]
    alpha = (ns - t0) / (t1 - t0)

    matrix = np.eye(4)
    matrix[:3, 3] = (1 - alpha) * a[:3, 3] + alpha * b[:3, 3]
    matrix[:3, :3] = Slerp(
        [0.0, 1.0],
        Rotation.from_matrix(np.stack([a[:3, :3], b[:3, :3]])),
    )(alpha).as_matrix()

    return matrix

class GTErrorNode(Node):
    def __init__(self):
        super().__init__("gt_error_node")

    if self.get_parameter("use_sim_time").value:
        raise ValueError("Live host timestamps require use_sim_time=false")

    self.frame = self.declare_parameter(
        "camera_frame", "zed_left_camera_optical_frame"
    ).value
    self.ready = self.declare_parameter("evaluation_enabled", True).value
    self.preliminary = self.declare_parameter("preliminary", True).value

    self.T_vc_c = checked_transform(
        np.array(
            self.declare_parameter(
                "camera_body_to_optical",
                CAMERA_AXES.ravel().tolist(),
            ).value
        ).reshape(4, 4)
    )
    self.T_vo_o = checked_transform(
        np.array(
            self.declare_parameter(
                "marker_body_to_mesh",
                np.eye(4).ravel().tolist(),
            ).value
        ).reshape(4, 4)
    )

    history_ns = int(positive(self, "history_s", 20.0) * 1e9)
    self.gap_ns = int(positive(self, "max_vicon_gap_s", 0.05) * 1e9)
    self.wait_s = positive(self, "sync_wait_s", 1.0)

    self.history = {
        name: PoseHistory(history_ns) for name in ("camera", "marker")
    }
    self.frames = {}
    self.frame_fault = False
    self.pending = deque()
    self.last_estimate = None
    self.last_log = 0.0
    self.matched = 0
    self.missed = 0

    self.gt_pub = self.create_publisher(
        PoseStamped, "/ground_truth/object_pose", 100
    )
    self.error_pub = self.create_publisher(
        DiagnosticArray, "/megapose/errors", 10
    )
    self.delta_pub = self.create_publisher(
        Vector3Stamped, "/megapose/translation_error", 10
    )

    qos = QoSProfile(
        depth=1000,
        reliability=ReliabilityPolicy.BEST_EFFORT,
    )

```

```

for name, parameter, default in [
    ("camera", "camera_vicon_topic", "/vicon/ZED_CAM/ZED_CAM"),
    (
        "marker",
        "marker_vicon_topic",
        "/vicon/MARKER_YFORWARD/MARKER_YFORWARD",
    ),
]:
    topic = self.declare_parameter(parameter, default).value
    self.create_subscription(
        TransformStamped,
        topic,
        lambda msg, key=name: self.on_vicon(key, msg),
        qos,
    )

    self.create_subscription(
        PoseStamped,
        "/megapose/object_pose",
        self.on_estimate,
        100,
    )

    self.create_timer(0.02, self.process)
    self.create_timer(1.0, self.idle_status)

    if self.preliminary:
        self.get_logger().warning(
            "Using metrics_postprocess.py frame assumptions; "
            "errors are preliminary."
        )

def report(self, state, ns=None, **values):
    diagnostic(
        self.error_pub,
        "megapose_vs_vicon",
        state,
        DiagnosticStatus.OK
        if state == "MATCHED"
        else DiagnosticStatus.WARN,
        time.time_ns() if ns is None else ns,
        matched=self.matched,
        missed=self.missed,
        preliminary=self.preliminary,
        **values,
    )

def idle_status(self):
    if not self.ready:
        self.report("EVALUATION_DISABLED")
    elif self.frame_fault:
        self.report("VICON_FRAME_MISMATCH")
    elif (
        self.last_estimate is None
        or time.monotonic() - self.last_estimate > 2.0
    ):
        self.report("WAITING_FOR_ESTIMATE")

def on_vicon(self, name, msg):
    ns = stamp_ns(msg.header.stamp)
    pair = (msg.header.frame_id, msg.child_frame_id)

    if ns <= 0 or not all(pair) or self.frame_fault:
        return

    if name in self.frames and self.frames[name] != pair:
        self.frame_fault = True
        self.report("VICON_FRAME_MISMATCH")
        return

    try:
        matrix = from_pose(
            msg.transform.translation,
            msg.transform.rotation,
        )

```

```

except ValueError:
    return

self.frames[name] = pair

if (
    len(self.frames) == 2
    and self.frames["camera"][0] != self.frames["marker"][0]
):
    self.frame_fault = True
    self.report("VICON_FRAME_MISMATCH")
    return

self.history[name].add(ns, matrix)

def on_estimate(self, msg):
    self.last_estimate = time.monotonic()

    if not self.ready or self.frame_fault:
        return

    ns = stamp_ns(msg.header.stamp)
    if ns <= 0 or msg.header.frame_id != self.frame:
        self.report("INVALID_ESTIMATE_FRAME")
        return

    try:
        matrix = from_pose(msg.pose.position, msg.pose.orientation)
    except ValueError:
        self.report("INVALID_ESTIMATE")
        return

    if len(self.pending) >= 200:
        self.pending.popleft()
        self.missed += 1

    self.pending.append((ns, matrix, time.monotonic()))

def process(self):
    if self.frame_fault:
        self.pending.clear()
        return

    while self.pending:
        ns, estimate, received = self.pending[0]

        try:
            camera = self.history["camera"].at(ns, self.gap_ns)
            marker = self.history["marker"].at(ns, self.gap_ns)
        except LookupError as error:
            if time.monotonic() - received < self.wait_s:
                break

            self.pending.popleft()
            self.missed += 1
            self.report("VICON_TIMEOUT", ns, reason=error)
            continue
        except ValueError as error:
            self.pending.popleft()
            self.missed += 1
            self.report("UNMATCHED", ns, reason=error)
            continue

        self.pending.popleft()

        # Object/mesh frame expressed in the camera optical frame.
        gt = np.linalg.inv(camera @ self.T_vc_c) @ marker @ self.T_vo_o

        delta = estimate[:3, 3] - gt[:3, 3]
        position_error = float(np.linalg.norm(delta))
        rotation_error = float(
            np.degrees(
                Rotation.from_matrix(
                    gt[:3, :3].T @ estimate[:3, :3]
                ).magnitude()
            )

```

```

    )
)

self.gt_pub.publish(pose_message(gt, ns, self.frame))

msg = Vector3Stamped()
set_stamp(msg.header, ns, self.frame)
msg.vector.x, msg.vector.y, msg.vector.z = map(float, delta)
self.delta_pub.publish(msg)

self.matched += 1
self.report(
    "MATCHED",
    ns,
    position_error_m=position_error,
    rotation_error_deg=rotation_error,
    estimate_age_s=(time.time_ns() - ns) / 1e9,
)

if time.monotonic() - self.last_log >= 1.0:
    self.get_logger().info(
        f"Position error: {position_error * 100:.2f} cm; "
        f"rotation error: {rotation_error:.2f} deg"
    )
    self.last_log = time.monotonic()

def main(args=None):
    rclpy.init(args=args)
    node = None

    try:
        node = GLErrorNode()
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        if node is not None:
            node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()

if __name__ == "__main__":
    main()

```

Replace `ros2_ws/src/mpose_live/setup.py` with:

python

```

from glob import glob

from setuptools import find_packages, setup

package_name = "mpose_live"

setup(
    name=package_name,
    version="0.0.0",
    packages=find_packages(exclude=["test"]),
    data_files=[
        (
            "share/ament_index/resource_index/packages",
            ["resource/" + package_name],
        ),
        ("share/" + package_name, ["package.xml"]),
        ("share/" + package_name + "/launch", glob("launch/*.launch.py")),
    ],
    install_requires=["setuptools"],
    zip_safe=True,
    maintainer="abhi",
    maintainer_email="coderadonis@gmail.com",
    description="MegaPose UDP bridge and live Vicon pose comparison",
    license="TODO: License declaration",
)

```

```

    extras_require={"test": ["pytest"]},
    entry_points={
        "console_scripts": [
            "mpose_bridge_node = mpose_live.mpose_bridge_node:main",
            "gt_error_node = mpose_live.gt_error_node:main",
        ],
    },
)

```

Ensure package.xml contains these dependencies inside <package>; add any missing entries:

xml

```

<buildtool_depend>ament_python</buildtool_depend>

<exec_depend>rclpy</exec_depend>
<exec_depend>geometry_msgs</exec_depend>
<exec_depend>diagnostic_msgs</exec_depend>
<exec_depend>python3-numpy</exec_depend>
<exec_depend>python3-scipy</exec_depend>
<exec_depend>launch</exec_depend>
<exec_depend>launch_ros</exec_depend>

```

Save this as ros2_ws/src/mpose_live/launch/demo.launch.py to start both nodes:

python

```

from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package="mpose_live",
            executable="mpose_bridge_node",
            output="screen",
            parameters=[{"use_sim_time": False}],
        ),
        Node(
            package="mpose_live",
            executable="gt_error_node",
            output="screen",
            parameters=[{"use_sim_time": False}],
        ),
    ])

```

Build and launch from a ROS terminal outside the mpose Conda environment:

bash

```

source /opt/ros/humble/setup.bash
cd /home/abhi/dev/ros2_ws

/usr/bin/python3 /usr/bin/colcon build --symlink-install --packages-select mpose_live

source install/setup.bash
ros2 launch mpose_live demo.launch.py

```

You can also run each independently in separate sourced terminals:

bash

```
ros2 run mpose_live mpose_bridge_node
```

bash

```
ros2 run mpose_live gt_error_node
```

Keep inference running separately in mpose, and stop check_udp.py before starting the bridge so port 5005 is available. The old ros_nodes.py is no longer used.

Both split modules passed import and synthetic checks for packet validation, timestamps, interpolation, and known pose errors. Live ROS launch remains untested here.

053 You

Sep 24, 2026 07:15 AM EDT

Have you performed this conversion in `gt_error_node.py` ?

```
def convert_to_cvframe(T_world_camera_body):
    """Convert the tracked camera-body pose to the OpenCV optical frame.
```

csharp

```
    Transform convention: T_A_B maps coordinates from frame B into frame A.
    The calibrated axis relationship is:
        optical X = body -X
        optical Y = body -Z
        optical Z = body -Y
    """
```

```
T_camera_body_camera_optical = np.array([
[-1.0, 0.0, 0.0, 0.0],
[ 0.0, 0.0, -1.0, 0.0],
[ 0.0, -1.0, 0.0, 0.0],
[ 0.0, 0.0, 0.0, 1.0],
], dtype=float)
return T_world_camera_body @ T_camera_body_camera_optical
```

054 Assistant

Sep 24, 2026 07:15 AM EDT

Yes—in the `gt_error_node.py` I provided, `CAMERA_AXES` is exactly your conversion matrix. It initializes `self.T_vc_c` through the `camera_body_to_optical` parameter.

The conversion happens in this line:

python

```
gt = np.linalg.inv(camera @ self.T_vc_c) @ marker @ self.T_vo_o
```

Written explicitly, that is:

python

```
# Same operation as your convert_to_cvframe(camera):
T_world_camera_optical = camera @ self.T_vc_c

# Express the tracked object's mesh pose in the camera optical frame:
T_world_object = marker @ self.T_vo_o
gt = np.linalg.inv(T_world_camera_optical) @ T_world_object
```

So **the conversion is already applied**; do not apply it again to the incoming Vicon camera pose. Like your function, the default assumes zero translation between the camera-body and optical-frame origins.

055 You

Sep 24, 2026 07:17 AM EDT

why is position error still 80 cm ?

056 Assistant • progress update

Sep 24, 2026 07:17 AM EDT

I'll check the frame conversion and error calculation against your current files. An 80 cm error could come from a frame-origin offset or the estimated pose itself; the axis conversion alone doesn't distinguish those.

057 Assistant • clarification

Sep 24, 2026 07:17 AM EDT

While the robot is stationary, what position (x, y, z) does each topic report? Run `ros2 topic echo /megapose/object_pose --once` and `ros2 topic echo /ground_truth/object_pose --once`, and paste both outputs. These will show which axis accounts for the 80 cm error.

058 Assistant • progress update

Sep 24, 2026 07:17 AM EDT

I found a possible scale mismatch: `start_demo.bash` loads `BaselinePointer.obj`, which is 20 cm long, while `PointerActual.obj` is 15 cm long. If the physical marker is 15 cm, the larger mesh could make MegaPose overestimate distance by roughly one third—about 80 cm at a 2.4 m distance.

059 Assistant • clarification

Sep 24, 2026 07:17 AM EDT

Is the physical pointer 15 cm or 20 cm along its long dimension, and are you running the command from `start_demo.bash` with `--mesh BaselinePointer.obj`? The repository contains both a 15 cm `PointerActual.obj` and a 20 cm `BaselinePointer.obj`.

060 You

Sep 24, 2026 07:19 AM EDT

Now write the `tbot` controller as we planed originally

061 Assistant • progress update

Sep 24, 2026 07:20 AM EDT

I'll check the original demo brief and add a TurtleBot controller that consumes the pose topic. Since the 80 cm discrepancy is unresolved, I'll include a dry-run mode so you can inspect velocity commands before enabling motion.

062 Assistant • clarification

Sep 24, 2026 07:20 AM EDT

With the camera fixed outside and the marker attached to the robot, what should the controller do first? The brief's image-centering rule assumed a different camera arrangement.

- Rotate to a chosen heading (recommended first)
- Drive to a chosen point on the floor

063 Assistant • progress update

Sep 24, 2026 07:21 AM EDT

The first milestone will be a relative heading turn: capture the robot's starting orientation, then use MegaPose feedback to turn it by a requested angle. This fits the fixed-camera setup without needing a camera-to-floor transform. I'll add speed limits, stale-pose and target-loss stops, plus start/stop services; Vicon will remain an independent check.

064 Assistant • progress update

Sep 24, 2026 07:24 AM EDT

The controller passed simulated turn and stop tests, including stale images, target loss, pose jumps, and inference restarts. It needs one mounting setting: the robot's upward direction expressed in the mesh frame. I've made that configurable; the default is mesh +Z. Dry-run mode publishes only a preview topic.

065 Assistant

Sep 24, 2026 07:26 AM EDT

For your **fixed camera + robot-mounted marker**, the first controller should turn the robot by a requested angle using the marker's **orientation**.

This version:

- Captures the current heading when you call start.
- Turns by target_yaw_deg, then stops.
- Stops on lost tracking, stale poses, orientation jumps, or timeout.
- Starts in **dry-run mode**, publishing commands only to a preview topic.
- Uses MegaPose directly; Vicon remains an independent check.

It does not use position yet, so the unresolved 80 cm translation discrepancy does not enter this control calculation.

The workspace is read-only, so save this as

ros2_ws/src/mpose_live/mpose_live/turtlebot_controller_node.py:

python

```
#!/usr/bin/env python3
"""Relative yaw control using a fixed camera and a robot-mounted marker."""

import math
import time
from collections import OrderedDict

import numpy as np
import rclpy
from diagnostic_msgs.msg import DiagnosticArray, DiagnosticStatus
from geometry_msgs.msg import PoseStamped, Twist
from rcl_interfaces.msg import ParameterDescriptor
from rclpy.node import Node
from scipy.spatial.transform import Rotation
from std_srvs.srv import Trigger

def stamp_ns(stamp):
    return stamp.sec * 1_000_000_000 + stamp.nanosec

def wrap(angle):
    return math.atan2(math.sin(angle), math.cos(angle))

def relative_yaw(reference, current, up):
    # Maps current mesh coordinates into the initial mesh frame.
    relative_rotation = reference.T @ current

    tilt = math.acos(
        float(np.clip(up @ (relative_rotation @ up), -1.0, 1.0))
    )

    # Choose an arbitrary horizontal direction perpendicular to robot up.
    helper = np.eye(3)[int(np.argmax(np.abs(up)))]
    forward = np.cross(up, helper)
```

```

forward /= np.linalg.norm(forward)

rotated = relative_rotation @ forward
yaw = math.atan2(
    float(up @ np.cross(forward, rotated)),
    float(forward @ rotated),
)
return yaw, tilt

class TurtleBotController(Node):
    def __init__(self):
        super().__init__("turtlebot_controller_node")

        if self.get_parameter("use_sim_time").value:
            raise ValueError("The live bridge requires use_sim_time=false")

        def parameter(name, default):
            return self.declare_parameter(
                name,
                default,
                ParameterDescriptor(read_only=True),
            ).value

        def positive(name, default):
            value = float(parameter(name, default))
            if not math.isfinite(value) or value <= 0:
                raise ValueError(f"{name} must be finite and positive")
            return value

        self.frame = parameter(
            "camera_frame", "zed_left_camera_optical_frame"
        )
        self.dry_run = parameter("dry_run", True)

        self.goal = math.radians(
            float(parameter("target_yaw_deg", 45.0))
        )
        if (
            not math.isfinite(self.goal)
            or abs(self.goal) > math.radians(170)
        ):
            raise ValueError("target_yaw_deg must be between -170 and 170")

        self.kp = positive("kp_yaw", 0.8)
        self.max_w = positive("max_angular_speed", 0.25)
        self.tolerance = math.radians(
            positive("yaw_tolerance_deg", 3.0)
        )
        self.max_tilt = math.radians(
            positive("max_tilt_deg", 20.0)
        )
        self.max_step = math.radians(
            positive("max_pose_step_deg", 35.0)
        )
        self.pose_age = positive("max_pose_age_s", 0.5)
        self.status_timeout = positive("status_timeout_s", 0.6)
        self.run_timeout = positive("run_timeout_s", 30.0)
        hz = positive("control_hz", 20.0)

        # Physical robot up (+base_link Z), expressed in the mesh frame.
        self.up = np.asarray(
            parameter("robot_up_in_marker", [0.0, 0.0, 1.0])
        )
        if (
            self.up.shape != (3,)
            or not np.isfinite(self.up).all()
            or np.linalg.norm(self.up) < 1e-9
        ):
            raise ValueError(
                "robot_up_in_marker must be a nonzero 3-vector"
            )
        self.up = self.up / np.linalg.norm(self.up)

        self.preview_pub = self.create_publisher(

```

```

        Twist, "~/command_preview", 1
    )

    command_topic = parameter("cmd_vel_topic", "/cmd_vel")
    self.cmd_pub = None
    if not self.dry_run:
        self.cmd_pub = self.create_publisher(
            Twist, command_topic, 1
        )

    self.poses = OrderedDict()
    self.last_pose_ns = 0
    self.previous_rotation = None

    self.status_received = None
    self.status_ns = 0
    self.status_image_ns = 0
    self.status_ok = False
    self.session = None

    self.reference = None
    self.active = False
    self.started = 0.0
    self.last_log = 0.0

    self.create_subscription(
        PoseStamped,
        "/megapose/object_pose",
        self.on_pose,
        10,
    )
    self.create_subscription(
        DiagnosticArray,
        "/megapose/status",
        self.on_status,
        10,
    )

    self.create_service(Trigger, "~/start", self.start)
    self.create_service(Trigger, "~/stop", self.stop)
    self.create_timer(1.0 / hz, self.tick)

    self.get_logger().info(
        f"Ready; dry_run={self.dry_run}, "
        f"target={math.degrees(self.goal):.1f} deg. "
        "Call ~/start to capture the initial heading and begin."
    )

def publish(self, angular=0.0):
    msg = Twist()
    msg.angular.z = float(angular)

    # linear.x remains zero: this version only turns in place.
    self.preview_pub.publish(msg)

    if self.cmd_pub is not None:
        self.cmd_pub.publish(msg)

def halt(self, reason):
    was_active = self.active
    self.active = False
    self.publish()

    if was_active:
        self.get_logger().warning(
            f"Stopped: {reason}. Call ~/start to restart."
        )

def on_pose(self, msg):
    ns = stamp_ns(msg.header.stamp)
    age = (time.time_ns() - ns) / 1e9

    if (
        msg.header.frame_id != self.frame
        or ns <= 0
    )

```

```

        or age < -0.05
        or age > self.pose_age
    ):
        self.halt("invalid frame or stale pose")
        return

    if ns <= self.last_pose_ns:
        return

    p = msg.pose.position
    q = msg.pose.orientation
    values = np.array([
        p.x, p.y, p.z,
        q.x, q.y, q.z, q.w,
    ])
    quaternion = values[3:]

    if (
        not np.isfinite(values).all()
        or p.z <= 0
        or abs(np.linalg.norm(quaternion) - 1.0) > 0.01
    ):
        self.halt("invalid pose")
        return

    rotation = Rotation.from_quat(quaternion).as_matrix()

    if self.active and self.previous_rotation is not None:
        step = Rotation.from_matrix(
            self.previous_rotation.T @ rotation
        ).magnitude()

        if step > self.max_step:
            self.halt("large orientation jump")
            self.poses.clear()
            self.last_pose_ns = ns
            self.previous_rotation = rotation
            return

    self.previous_rotation = rotation
    self.last_pose_ns = ns
    self.poses[ns] = rotation

    while len(self.poses) > 20:
        self.poses.popitem(last=False)

def on_status(self, msg):
    ns = stamp_ns(msg.header.stamp)
    age = (time.time_ns() - ns) / 1e9

    if (
        ns <= 0
        or age < -0.05
        or age > self.status_timeout
    ):
        self.status_ok = False
        self.halt("stale bridge status")
        return

    if ns < self.status_ns:
        return

    entries = [
        status for status in msg.status
        if status.name == "megapose_stream"
    ]
    if len(entries) != 1:
        self.status_ok = False
        self.halt("missing bridge status")
        return

    status = entries[0]
    values = {item.key: item.value for item in status.values}
    session = values.get("session_id", "")

```

```

    valid = (
        status.message == "TRACKING"
        and status.level == DiagnosticStatus.OK
        and values.get("valid", "").lower() == "true"
        and session not in ("", "None")
    )

    try:
        image_ns = int(values.get("image_time_ns", "0"))
    except ValueError:
        image_ns = 0

    valid = valid and image_ns > 0

    if self.session is not None and session != self.session:
        self.halt("inference session changed")
        self.poses.clear()

    self.session = session
    self.status_ns = ns
    self.status_received = time.monotonic()
    self.status_image_ns = image_ns
    self.status_ok = valid

    if not valid:
        self.halt(f"bridge reports {status.message}")

def current_pose(self):
    if not self.status_ok or self.status_received is None:
        return None, "TRACKING status unavailable"

    if (
        time.monotonic() - self.status_received
        > self.status_timeout
    ):
        return None, "bridge status timeout"

    age = (time.time_ns() - self.status_image_ns) / 1e9
    if age < -0.05 or age > self.pose_age:
        return None, "source image is stale"

    rotation = self.poses.get(self.status_image_ns)
    if rotation is None:
        return None, "WAITING_FOR_PAIR"

    return rotation, ""

def start(self, request, response):
    if self.active:
        response.success = False
        response.message = (
            "Already active; stop before starting another turn."
        )
        return response

    rotation, reason = self.current_pose()
    if rotation is None:
        response.success = False
        response.message = reason
        return response

    self.reference = rotation.copy()
    self.previous_rotation = rotation.copy()
    self.started = time.monotonic()
    self.active = True

    response.success = True
    response.message = (
        f"Target: {math.degrees(self.goal):.1f} deg "
        f"from current heading; dry_run={self.dry_run}"
    )
    self.get_logger().info(response.message)
    return response

def stop(self, request, response):

```

```

        self.halt("stop service")
        response.success = True
        response.message = "Stopped."
        return response

def tick(self):
    if not self.active:
        self.publish()
        return

    if time.monotonic() - self.started > self.run_timeout:
        self.halt("run timeout")
        return

    rotation, reason = self.current_pose()
    if rotation is None:
        if reason == "WAITING_FOR_PAIR":
            # Pose and status topics may arrive in either order.
            self.publish()
        else:
            self.halt(reason)
            return

    yaw, tilt = relative_yaw(
        self.reference, rotation, self.up
    )
    if tilt > self.max_tilt:
        self.halt(
            "excessive tilt or inconsistent marker orientation"
        )
        return

    error = wrap(self.goal - yaw)

    if abs(error) <= self.tolerance:
        self.active = False
        self.publish()
        self.get_logger().info("Target heading reached.")
        return

    angular = float(
        np.clip(self.kp * error, -self.max_w, self.max_w)
    )
    self.publish(angular)

    if time.monotonic() - self.last_log >= 1.0:
        self.get_logger().info(
            f"yaw={math.degrees(yaw):.1f} deg, "
            f"error={math.degrees(error):.1f} deg, "
            f"angular.z={angular:.3f} rad/s, "
            f"dry_run={self.dry_run}"
        )
        self.last_log = time.monotonic()

def main(args=None):
    rclpy.init(args=args)
    node = None

    try:
        node = TurtleBotController()
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        if node is not None:
            if rclpy.ok():
                node.halt("shutdown")
                node.destroy_node()

        if rclpy.ok():
            rclpy.shutdown()

if __name__ == "__main__":

```

```
main()
```

Add this entry to console_scripts in setup.py (/home/abhi/dev/ros2_ws/src/mpose_live/setup.py):

```
python
```

```
"turtlebot_controller_node = mpose_live.turtlebot_controller_node:main",
```

Your current package.xml (/home/abhi/dev/ros2_ws/src/mpose_live/package.xml) still lacks runtime dependencies. Add these inside <package>, skipping any already present:

```
xml
```

```
<buildtool_depend>ament_python</buildtool_depend>

<exec_depend>rclpy</exec_depend>
<exec_depend>rcl_interfaces</exec_depend>
<exec_depend>geometry_msgs</exec_depend>
<exec_depend>diagnostic_msgs</exec_depend>
<exec_depend>std_srvs</exec_depend>
<exec_depend>python3-numpy</exec_depend>
<exec_depend>python3-scipy</exec_depend>
<exec_depend>launch</exec_depend>
<exec_depend>launch_ros</exec_depend>
```

Check the mounting parameter before enabling motion. robot_up_in_marker means the robot's upward direction expressed in the **MegaPose mesh frame**, not the Vicon frame:

Mesh axis pointing physically upward	Parameter
+Z	[0.0, 0.0, 1.0]
+Y	[0.0, 1.0, 0.0]
-Z	[0.0, 0.0, -1.0]

Keep the camera fixed throughout each turn. Positive target angles mean counterclockwise viewed from above.

Build from your ROS terminal:

```
bash
```

```
source /opt/ros/humble/setup.bash
cd /home/abhi/dev/ros2_ws

/usr/bin/python3 /usr/bin/colcon build \
  --symlink-install --packages-select mpose_live

source install/setup.bash
```

With inference and the bridge running, start the controller in dry-run mode:

```
bash
```

```
ros2 run mpose_live turtlebot_controller_node --ros-args \
  -p dry_run:=true \
  -p target_yaw_deg:=45.0 \
  -p robot_up_in_marker:="[0.0, 0.0, 1.0]"
```

Start a turn calculation:

```
bash
```

```
ros2 service call /turtlebot_controller_node/start std_srvs/srv/Trigger "{}"
```

Inspect its proposed commands:

```
bash
```

```
ros2 topic echo /turtlebot_controller_node/command_preview
```

During dry-run, physically rotating the robot counterclockwise should increase the reported yaw and reduce the positive target error.

To enable motor commands, restart the controller with `dry_run:=false` and your robot's actual velocity topic. Your existing experiment launch uses `/TB3_3/cmd_vel` with `geometry_msgs/msg/Twist`:

```
bash
```

```
ros2 run mpose_live turtlebot_controller_node --ros-args \
  -p dry_run:=false \
  -p cmd_vel_topic:=/TB3_3/cmd_vel \
  -p target_yaw_deg:=30.0 \
  -p robot_up_in_marker:="[0.0, 0.0, 1.0]"
```

Call start again to begin. Stop at any time with:

```
bash
```

```
ros2 service call /turtlebot_controller_node/stop std_srvs/srv/Trigger "{}"
```

Each start captures a new reference heading. Tracking loss stops the run and requires another start. Use only one active motion controller; the robot's own velocity timeout must handle a controller crash or connection loss.

Synthetic tests passed for turn direction, camera/mount geometry, speed limits, target loss, stale data, orientation jumps, and stopping. This has **not been tested on the physical robot**.

066 You

Sep 24, 2026 07:37 AM EDT

I want you to write the tbot controller as we planned

067 Assistant • progress update

Sep 24, 2026 07:37 AM EDT

I'll check the current controller against the intended behavior. The last version only turned through a relative angle; I need to confirm whether you want the robot to drive to a target position as well.

068 Assistant • clarification

Sep 24, 2026 07:37 AM EDT

What should the planned controller make the robot do, with the camera fixed outside and the marker attached to the robot?

- Drive to a target floor position, then stop
- Turn to a target heading only
- Follow a sequence of floor waypoints

069 Assistant • progress update

Sep 24, 2026 07:38 AM EDT

There's no controller file saved in the package yet. I verified the geometry for using the robot's starting pose as a local reference, which can support either turning or driving to a point without a camera-to-world calibration. I'm waiting on the behavior choice before writing another version. This workspace is still read-only, so I can provide the file contents but cannot save them here.

070 Assistant

Sep 24, 2026 07:39 AM EDT

The previous version only turned through a relative angle. With your **fixed external camera and marker attached to the robot**, should the planned controller:

1. **Drive to a target floor position and stop**
2. Follow a sequence of waypoints
3. Turn to a target heading only

Which behavior did you intend? I don't want to repeat the wrong controller.

071 You

Sep 24, 2026 07:45 AM EDT

I want you to write the tbot controller as we planned. We shall drive to a target floor position then stop

072 Assistant • progress update

Sep 24, 2026 07:45 AM EDT

I'll write the controller to drive to a target floor position and stop. I'll express the target relative to the robot's starting pose—X forward, Y left—and make the marker-to-robot mounting transform configurable. It will start in dry-run mode while the position error is unresolved.

073 Assistant • progress update

Sep 24, 2026 07:53 AM EDT

The controller now turns toward the goal, drives forward when aligned, and stops within 5 cm after three distinct pose samples confirm arrival. Simulated runs passed for goals ahead, to the side, and behind the robot. I also checked target-loss and stale-data stops. The remaining setup inputs are the marker's mounting transform and the robot's velocity topic.

074 Assistant

Sep 24, 2026 07:56 AM EDT

This controller **drives to a target floor position and stops**:

1. Capture the robot's starting pose.
2. Rotate toward the target.
3. Drive forward while correcting heading.
4. Stop within the goal tolerance.

The goal uses the robot's starting frame: **X forward, Y left, metres**. For example, (0.5, 0.25) means 50 cm forward and 25 cm left of its starting position.

The workspace is read-only, so save the following as

ros2_ws/src/mpose_live/mpose_live/turtlebot_controller_node.py.

python

```
#!/usr/bin/env python3
"""Drive to a floor goal using a fixed camera and a robot-mounted marker."""

import math
import time
from collections import OrderedDict
```

```

import numpy as np
import rclpy
from diagnostic_msgs.msg import DiagnosticArray, DiagnosticStatus
from geometry_msgs.msg import PoseStamped, Twist
from rcl_interfaces.msg import ParameterDescriptor
from rclpy.node import Node
from scipy.spatial.transform import Rotation
from std_srvs.srv import Trigger

def stamp_ns(stamp):
    return stamp.sec * 1_000_000_000 + stamp.nanosec

def wrap(angle):
    return math.atan2(math.sin(angle), math.cos(angle))

def planar_command(
    pose,
    goal,
    kp_distance,
    kp_heading,
    max_v,
    max_w,
    heading_gate,
    tolerance,
):
    x, y = pose[:2, 3]
    yaw = math.atan2(pose[1, 0], pose[0, 0])

    dx, dy = goal - np.array([x, y])
    distance = math.hypot(dx, dy)
    error = wrap(math.atan2(dy, dx) - yaw)

    if distance <= tolerance:
        return 0.0, 0.0, distance, error

    angular = float(np.clip(kp_heading * error, -max_w, max_w))

    # Turn toward the goal before driving forward.
    linear = 0.0
    if abs(error) <= heading_gate:
        linear = min(max_v, kp_distance * distance) * math.cos(error)

    return linear, angular, distance, error

class TurtleBotController(Node):
    def __init__(self):
        super().__init__("turtlebot_controller_node")

        if self.get_parameter("use_sim_time").value:
            raise ValueError("The live bridge requires use_sim_time=false")

        def parameter(name, default):
            return self.declare_parameter(
                name,
                default,
                ParameterDescriptor(read_only=True),
            ).value

        def positive(name, default):
            value = float(parameter(name, default))
            if not math.isfinite(value) or value <= 0:
                raise ValueError(f"{name} must be finite and positive")
            return value

        self.frame = parameter(
            "camera_frame", "zed_left_camera_optical_frame"
        )
        self.dry_run = parameter("dry_run", True)

        self.goal = np.asarray([
            parameter("goal_x_m", 0.5),

```

```

        parameter("goal_y_m", 0.0),
    ], dtype=float)

    max_goal = positive("max_goal_distance_m", 2.0)
    if (
        not np.isfinite(self.goal).all()
        or np.linalg.norm(self.goal) > max_goal
    ):
        raise ValueError(
            "Goal must be finite and within max_goal_distance_m"
        )

    self.kp_v = positive("kp_distance", 0.4)
    self.kp_w = positive("kp_heading", 1.0)
    self.max_v = positive("max_linear_speed", 0.08)
    self.max_w = positive("max_angular_speed", 0.3)
    self.tolerance = positive("goal_tolerance_m", 0.05)

    self.heading_gate = math.radians(
        positive("drive_heading_limit_deg", 20.0)
    )
    if self.heading_gate >= math.pi / 2:
        raise ValueError("drive_heading_limit_deg must be below 90")

    self.max_tilt = math.radians(
        positive("max_tilt_deg", 20.0)
    )
    self.max_step = math.radians(
        positive("max_rotation_step_deg", 35.0)
    )
    self.max_position_step = positive(
        "max_position_step_m", 0.25
    )
    self.max_height = positive("max_height_change_m", 0.15)
    self.max_travel = positive("max_distance_from_start_m", 3.0)

    if np.linalg.norm(self.goal) > self.max_travel:
        raise ValueError("Goal exceeds max_distance_from_start_m")

    self.pose_age = positive("max_pose_age_s", 0.5)
    self.status_timeout = positive("status_timeout_s", 0.6)
    self.run_timeout = positive("run_timeout_s", 60.0)
    hz = positive("control_hz", 20.0)

    # T_base_marker:
    # Mesh origin and mesh axes expressed in the robot base frame.
    # Robot base convention: X forward, Y left, Z up.
    xyz = np.asarray(
        parameter("marker_in_base_xyz", [0.0, 0.0, 0.0])
    )
    rpy = np.asarray(
        parameter("marker_in_base_rpy_deg", [0.0, 0.0, 0.0])
    )

    if (
        xyz.shape != (3,)
        or rpy.shape != (3,)
        or not np.isfinite(xyz).all()
        or not np.isfinite(rpy).all()
    ):
        raise ValueError("Mounting parameters must be finite 3-vectors")

    mount = np.eye(4)
    mount[:3, :3] = Rotation.from_euler(
        "xyz", rpy, degrees=True
    ).as_matrix()
    mount[:3, 3] = xyz

    self.T_marker_base = np.linalg.inv(mount)

    self.preview_pub = self.create_publisher(
        Twist, "~/command_preview", 1
    )

    command_topic = parameter("cmd_vel_topic", "/cmd_vel")

```

```

self.cmd_pub = None
if not self.dry_run:
    self.cmd_pub = self.create_publisher(
        Twist, command_topic, 1
    )

self.poses = OrderedDict()
self.last_pose_ns = 0
self.previous_pose = None

self.status_received = None
self.status_ns = 0
self.status_image_ns = 0
self.status_ok = False
self.session = None

self.reference = None
self.active = False
self.started = 0.0
self.last_log = 0.0
self.goal_hits = 0
self.last_goal_stamp = 0

self.create_subscription(
    PoseStamped,
    "/megapose/object_pose",
    self.on_pose,
    10,
)
self.create_subscription(
    DiagnosticArray,
    "/megapose/status",
    self.on_status,
    10,
)

self.create_service(Trigger, "~/start", self.start)
self.create_service(Trigger, "~/stop", self.stop)
self.create_timer(1.0 / hz, self.tick)

self.get_logger().info(
    f"Ready; dry_run={self.dry_run}, "
    f"goal={self.goal.tolist()} m. "
    "Goal axes at start: X forward, Y left. "
    "Call ~/start to begin."
)

def publish(self, linear=0.0, angular=0.0):
    msg = Twist()
    msg.linear.x = float(linear)
    msg.angular.z = float(angular)

    self.preview_pub.publish(msg)
    if self.cmd_pub is not None:
        self.cmd_pub.publish(msg)

def halt(self, reason):
    was_active = self.active
    self.active = False
    self.publish()

    if was_active:
        self.get_logger().warning(
            f"Stopped: {reason}. Call ~/start to restart."
        )

def on_pose(self, msg):
    ns = stamp_ns(msg.header.stamp)
    age = (time.time_ns() - ns) / 1e9

    if (
        msg.header.frame_id != self.frame
        or ns <= 0
        or age < -0.05
        or age > self.pose_age
    )

```

```

    ):
        self.halt("invalid frame or stale pose")
        return

    if ns <= self.last_pose_ns:
        return

    p = msg.pose.position
    q = msg.pose.orientation

    values = np.array([
        p.x, p.y, p.z,
        q.x, q.y, q.z, q.w,
    ])
    quaternion = values[3:]

    if (
        not np.isfinite(values).all()
        or p.z <= 0
        or abs(np.linalg.norm(quaternion) - 1.0) > 0.01
    ):
        self.halt("invalid pose")
        return

    T_camera_marker = np.eye(4)
    T_camera_marker[:3, :3] = Rotation.from_quat(
        quaternion
    ).as_matrix()
    T_camera_marker[:3, 3] = values[:3]

    # Convert the estimated mesh pose to the robot base pose.
    pose = T_camera_marker @ self.T_marker_base

    if self.active and self.previous_pose is not None:
        angle_step = Rotation.from_matrix(
            self.previous_pose[:3, :3].T @ pose[:3, :3]
        ).magnitude()

        position_step = np.linalg.norm(
            pose[:3, 3] - self.previous_pose[:3, 3]
        )

        if (
            angle_step > self.max_step
            or position_step > self.max_position_step
        ):
            self.halt("large pose jump")
            self.poses.clear()
            self.last_pose_ns = ns
            self.previous_pose = pose
            return

    self.previous_pose = pose
    self.last_pose_ns = ns
    self.poses[ns] = pose

    while len(self.poses) > 20:
        self.poses.popitem(last=False)

def on_status(self, msg):
    ns = stamp_ns(msg.header.stamp)
    age = (time.time_ns() - ns) / 1e9

    if (
        ns <= 0
        or age < -0.05
        or age > self.status_timeout
    ):
        self.status_ok = False
        self.halt("stale bridge status")
        return

    if ns < self.status_ns:
        return

```

```

entries = [
    status for status in msg.status
    if status.name == "megapose_stream"
]
if len(entries) != 1:
    self.status_ok = False
    self.halt("missing bridge status")
    return

status = entries[0]
values = {
    item.key: item.value for item in status.values
}
session = values.get("session_id", "")

valid = (
    status.message == "TRACKING"
    and status.level == DiagnosticStatus.OK
    and values.get("valid", "").lower() == "true"
    and session not in ("", "None")
)

try:
    image_ns = int(values.get("image_time_ns", "0"))
except ValueError:
    image_ns = 0

valid = valid and image_ns > 0

if self.session is not None and session != self.session:
    self.halt("inference session changed")
    self.poses.clear()

self.session = session
self.status_ns = ns
self.status_received = time.monotonic()
self.status_image_ns = image_ns
self.status_ok = valid

if not valid:
    self.halt(f"bridge reports {status.message}")

def current_pose(self):
    if not self.status_ok or self.status_received is None:
        return None, "TRACKING status unavailable"

    if (
        time.monotonic() - self.status_received
        > self.status_timeout
    ):
        return None, "bridge status timeout"

    age = (time.time_ns() - self.status_image_ns) / 1e9
    if age < -0.05 or age > self.pose_age:
        return None, "source image is stale"

    pose = self.poses.get(self.status_image_ns)
    if pose is None:
        return None, "WAITING_FOR_PAIR"

    return pose, ""

def start(self, request, response):
    if self.active:
        response.success = False
        response.message = (
            "Already active; stop before starting another run."
        )
        return response

    pose, reason = self.current_pose()
    if pose is None:
        response.success = False
        response.message = reason
        return response

```

```

# Starting base frame becomes the local navigation frame.
self.reference = np.linalg.inv(pose)
self.previous_pose = pose.copy()

self.goal_hits = 0
self.last_goal_stamp = 0
self.started = time.monotonic()
self.active = True

response.success = True
response.message = (
    f"Goal: X={self.goal[0]:.2f} m forward, "
    f"Y={self.goal[1]:.2f} m left "
    f"from the current base pose; dry_run={self.dry_run}"
)
self.get_logger().info(response.message)
return response

def stop(self, request, response):
    self.halt("stop service")
    response.success = True
    response.message = "Stopped."
    return response

def tick(self):
    if not self.active:
        self.publish()
        return

    if time.monotonic() - self.started > self.run_timeout:
        self.halt("run timeout")
        return

    pose, reason = self.current_pose()
    if pose is None:
        if reason == "WAITING_FOR_PAIR":
            # Pose and status can arrive in either order.
            self.publish()
        else:
            self.halt(reason)
        return

    # Fixed camera: current base expressed in its starting frame.
    local = self.reference @ pose

    tilt = math.acos(
        float(np.clip(local[2, 2], -1.0, 1.0))
    )
    if (
        tilt > self.max_tilt
        or abs(local[2, 3]) > self.max_height
    ):
        self.halt("inconsistent floor plane or mounting transform")
        return

    if np.linalg.norm(local[:2, 3]) > self.max_travel:
        self.halt("outside distance limit")
        return

    linear, angular, distance, error = planar_command(
        local,
        self.goal,
        self.kp_v,
        self.kp_w,
        self.max_v,
        self.max_w,
        self.heading_gate,
        self.tolerance,
    )

    if distance <= self.tolerance:
        self.publish()

    # Require three distinct source images, not three timer ticks.
    if self.status_image_ns != self.last_goal_stamp:

```

```

        self.goal_hits += 1
        self.last_goal_stamp = self.status_image_ns

    if self.goal_hits >= 3:
        self.active = False
        self.get_logger().info("Goal reached. Stopped.")
        return

    self.goal_hits = 0
    self.publish(linear, angular)

    if time.monotonic() - self.last_log >= 1.0:
        yaw = math.atan2(local[1, 0], local[0, 0])
        self.get_logger().info(
            f"x={local[0, 3]:.3f}, y={local[1, 3]:.3f} m, "
            f"yaw={math.degrees(yaw):.1f} deg, "
            f"distance={distance:.3f} m, "
            f"heading_error={math.degrees(error):.1f} deg, "
            f"v={linear:.3f} m/s, w={angular:.3f} rad/s, "
            f"dry_run={self.dry_run}"
        )
        self.last_log = time.monotonic()

def main(args=None):
    rclpy.init(args=args)
    node = None

    try:
        node = TurtleBotController()
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        if node is not None:
            if rclpy.ok():
                node.halt("shutdown")
                node.destroy_node()

        if rclpy.ok():
            rclpy.shutdown()

if __name__ == "__main__":
    main()

```

Add this entry to console_scripts in setup.py (/home/abhi/dev/ros2_ws/src/mpose_live/setup.py):

python

```
"turtlebot_controller_node = mpose_live.turtlebot_controller_node:main",
```

Ensure package.xml (/home/abhi/dev/ros2_ws/src/mpose_live/package.xml) includes these dependencies:

xml

```

<buildtool_depend>ament_python</buildtool_depend>

<exec_depend>rclpy</exec_depend>
<exec_depend>rcl_interfaces</exec_depend>
<exec_depend>geometry_msgs</exec_depend>
<exec_depend>diagnostic_msgs</exec_depend>
<exec_depend>std_srvs</exec_depend>
<exec_depend>python3-numpy</exec_depend>
<exec_depend>python3-scipy</exec_depend>
<exec_depend>launch</exec_depend>
<exec_depend>launch_ros</exec_depend>

```

Set the marker mounting transform correctly. These parameters describe the **MegaPose mesh frame relative to the robot base**, not the Vicon marker frame:

- marker_in_base_xyz: mesh origin in robot coordinates, metres.
- marker_in_base_rpy_deg: mesh orientation relative to the robot, roll/pitch/yaw in degrees.

The zero defaults assume both frames coincide. For example, `[0.0, 0.0, 0.2]` describes a mesh origin directly 20 cm above the base origin; use your actual measurements. Correct mounting axes make the local navigation plane correspond to the floor.

Build in your ROS environment:

bash

```
source /opt/ros/humble/setup.bash
cd /home/abhi/dev/ros2_ws

/usr/bin/python3 /usr/bin/colcon build \
  --symlink-install --packages-select mpose_live

source install/setup.bash
```

Run alongside the inference and bridge nodes. This example previews a goal 50 cm forward and 25 cm left:

bash

```
ros2 run mpose_live turtlebot_controller_node --ros-args \
  -p dry_run:=true \
  -p goal_x_m:=0.5 \
  -p goal_y_m:=0.25
```

Start:

bash

```
ros2 service call /turtlebot_controller_node/start std_srvs/srv/Trigger "{}"
```

Inspect proposed velocities:

bash

```
ros2 topic echo /turtlebot_controller_node/command_preview
```

Stop:

bash

```
ros2 service call /turtlebot_controller_node/stop std_srvs/srv/Trigger "{}"
```

For actual driving, restart with `-p dry_run:=false`, your measured mounting parameters, and `-p cmd_vel_topic:=/TB3_3/cmd_vel` if that is your robot's Twist topic. Each start defines a new goal relative to the current robot pose. Tracking loss requires another explicit start.

Keep dry-run enabled until the 80 cm error is understood. Using a relative starting frame does not correct mesh-scale errors. This controller also assumes a fixed camera and clear floor; it has no obstacle avoidance. The base needs its own command timeout for connection loss.

Synthetic tests passed for goal convergence, mounting transforms, velocity limits, stale data, tracking loss, pose jumps, and stopping. No physical robot test was performed.

075 You

Sep 24, 2026 07:56 AM EDT

I want you to write the tbot controller as we planned. We shall drive to a target floor position then stop

076 Assistant

Sep 24, 2026 07:56 AM EDT

Understood: **drive to a target floor position, then stop.**

The complete controller is prepared, but **I haven't saved it into your package because this session has read-only filesystem access.** It belongs at:

```
/home/abhi/dev/ros2_ws/src/mpose_live/mpose_live/turtlebot_controller_node.py
```

Please enable workspace write access so I can save the controller and update its ROS entry point and dependencies directly. No further behavior clarification is needed.

077 You

Sep 28, 2026 01:26 PM EDT

Have a look at this ros package and tell me what parts of the flowchart has been prepared ?

078 Assistant • progress update

Sep 28, 2026 01:27 PM EDT

I'll inspect the ROS package and map what's implemented to each block in the flowchart, including any gaps in how the nodes connect.

079 Assistant

Sep 28, 2026 01:28 PM EDT

I inspected /home/abhi/dev/ros2_ws/src/mpose_live. **The inference-to-ROS path is prepared. Ground-truth comparison is implemented but disabled in the launch file. The TurtleBot controller is still missing from the package.**

Flowchart component	Status	What exists
Live camera → MegaPose	Implemented	rt_live.py (/home/abhi/dev/mpose_rt/rt_live.py:266) captures ZED images through OpenCV and runs inference outside ROS.
Local pose packets	Implemented	Sends pose, tracking status, and image timestamp to UDP 127.0.0.1:5005.
mpose_bridge_node	Implemented	mpose_bridge_node.py (/home/abhi/dev/ros2_ws/src/mpose_live/mpose_live/mpose_bridge_node.py:194) receives packets and publishes /megapose/object_pose and /megapose/status.
Camera and marker Vicon topics	External dependency	The comparison node subscribes to /vicon/ZED_CAM/ZED_CAM and /vicon/MARKER_YFORWARD/MARKER_YFORWARD. Their publishers must run separately.
gt_relative_pose_node	Implemented within the combined node	gt_error_node.py (/home/abhi/dev/ros2_ws/src/mpose_live/mpose_live/gt_error_node.py:344) interpolates Vicon poses at the estimated pose's image timestamp and applies the optical-frame conversion.
pose_error_node	Implemented within the combined node	Computes translation and rotation errors against the synchronized ground truth.
Live errors	Implemented	Publishes /ground_truth/object_pose, /megapose/errors, and /megapose/translation_error; also logs errors.
Error CSV	Not implemented	The inference script saves estimated poses to CSV, but the ROS comparison node does not save error CSVs.
turtlebot_controller_node	Not present	The controller previously provided in chat has not been saved or registered in this package.

Flowchart component	Status	What exists
Controller → /cmd_vel	Not implemented	No node in this package currently publishes robot velocity commands.

Two integration gaps are visible:

- **The current launch starts only the bridge.** The `gt_error_node` block is commented out in `demo.launch.py` (/home/abhi/dev/ros2_ws/src/mpose_live/launch/demo.launch.py:13).

- **Dependency declarations are incomplete.** `package.xml` (/home/abhi/dev/ros2_ws/src/mpose_live/package.xml:10) lists test dependencies but lacks the runtime dependencies used by the nodes.

The GT comparison still uses the preliminary mounting assumptions. This inspection confirms what is implemented on disk; it does not establish that the live comparison is accurate or that the earlier 80 cm discrepancy is resolved.

080 You

Sep 28, 2026 01:36 PM EDT

How am I supposed to run `rt_live.py` ?

081 Assistant

Sep 28, 2026 01:36 PM EDT

Run it in the `mpose` **Conda environment**:

bash

```
conda activate mpose
cd /home/abhi/dev/mpose_rt

python rt_live.py \
  --project-root /home/abhi/dev/mpose_rt \
  --cam_source 0 \
  --mesh PointerActual.obj \
  --cam_file zed_1080p_raw_calib.json \
  --fps 15 \
  --udp-port 5005
```

This opens the ZED through OpenCV, runs MegaPose, displays the results, and sends pose packets to 127.0.0.1:5005. It can run without ROS.

To receive those packets in ROS, open a **separate terminal outside the `mpose` environment**:

bash

```
source /opt/ros/humble/setup.bash
source /home/abhi/dev/ros2_ws/install/setup.bash

ros2 run mpose_live mpose_bridge_node
```

Stop `check_udp.py` first if it is using port 5005.

In the inference window, press `r` to reset tracking or `q` / **Escape** to quit.

082 You

Sep 28, 2026 01:37 PM EDT

`nvidia-smi` is no longer working

083 Assistant • progress update

Sep 28, 2026 01:37 PM EDT

I'll check the exact error, whether the NVIDIA driver is loaded, and whether the GPU is visible to the system.

084 Assistant • clarification

Sep 28, 2026 01:37 PM EDT

What exact output do you get when you run `nvidia-smi` in your normal terminal, and did this start after a reboot, system update, or suspend/resume? My sandbox cannot see `/dev/nvidia*`, so I need to distinguish a host problem from sandbox restrictions.

085 Assistant • progress update

Sep 28, 2026 01:38 PM EDT

The GPU is detected, and the loaded driver and NVIDIA library both report version `580.173.02`. The kernel log also shows a failed GPU runtime-suspend operation, and its power status is error. That points toward a driver or power-management problem, though I still need your normal-terminal output because GPU device access is restricted in this sandbox.

086 Assistant

Sep 28, 2026 01:38 PM EDT

The GPU is detected and its driver is loaded, but I found a **GPU power-management error**:

text

```
nvidia 0000:01:00.0: can't suspend
(nv_pmops_runtime_suspend [nvidia] returned -5)
```

The GPU's runtime power status also reports error. The loaded driver and NVIDIA library both match **580.173.02**, so there's no obvious version mismatch.

This makes runtime power management a likely suspect, although the exact cause isn't confirmed. [NVIDIA documents this power-management mechanism here](#).

If `nvidia-smi` fails in your normal terminal too, save your work and reboot, then test before starting inference:

bash

```
nvidia-smi
```

If it works, check MegaPose's environment:

bash

```
conda activate mpose
python -c "import torch; print('CUDA available:', torch.cuda.is_available())"
```

If it still fails, paste the exact terminal error. My sandbox cannot access `/dev/nvidia*`, so its own `nvidia-smi` failure alone cannot establish what is happening in your normal session.

087 You

Sep 28, 2026 01:38 PM EDT

Reply to: What exact output do you get when you run `nvidia-smi` in your normal terminal, and did this start after a reboot, system update, or suspend/resume? My sandbox cannot see `/dev/nvidia*`, so I need to distinguish a host problem from sandbox restrictions.

```
ls /dev/nvidia
nvidia0 nvidiactl nvidia-uvm
nvidia-caps/ nvidia-modeset nvidia-uvm-tools
```

088 Assistant

Sep 28, 2026 01:38 PM EDT

Those device nodes **exist on your host**; their absence in my checks was a sandbox restriction. Their presence doesn't confirm that the GPU is responding.

Please paste the full output of:

bash

```
/usr/bin/nvidia-smi
cat /sys/bus/pci/devices/0000:01:00.0/power/runtime_status
```

Also mention whether you have rebooted since the failure started. This will help distinguish the power-management fault from a different driver error.

089 You

Sep 28, 2026 01:39 PM EDT

run the command yourself

090 Assistant • progress update

Sep 28, 2026 01:39 PM EDT

I'll run both commands and check what this session can see.

091 Assistant • progress update

Sep 28, 2026 01:39 PM EDT

The sandbox still hides the GPU device files. I'll rerun `nvidia-smi` outside it to check the host directly.

092 Assistant

Sep 28, 2026 01:40 PM EDT

I ran it **outside the sandbox**. The host reports:

text

```
Unable to determine the device handle for GPU0: 0000:01:00.0: Unknown Error
No devices were found
```

The GPU's runtime power status is:

text

```
error
```

This confirms a host GPU/driver problem. The loaded driver and NVIDIA library match, but the kernel recorded a failed GPU power-management operation.

Save your work and reboot as the next recovery step. A driver reinstall isn't justified by the evidence yet.

093 You

Sep 28, 2026 01:45 PM EDT

How am I supposed to run this ros node?

094 Assistant • progress update

Sep 28, 2026 01:45 PM EDT

I'll check which nodes are currently registered in `mpose_live` and how the launch file starts them.

095 Assistant

Sep 28, 2026 01:45 PM EDT

For the **MegaPose ROS bridge**, use a terminal outside the `mpose` Conda environment:

```
bash
```

```
source /opt/ros/humble/setup.bash
source /home/abhi/dev/ros2_ws/install/setup.bash

ros2 run mpose_live mpose_bridge_node
```

Keep `rt_live.py` running separately in the `mpose` environment. Stop `check_udp.py` first so the bridge can bind port 5005.

To run the **Vicon error node**, use another terminal:

```
bash
```

```
source /opt/ros/humble/setup.bash
source /home/abhi/dev/ros2_ws/install/setup.bash

ros2 run mpose_live gt_error_node
```

The Vicon camera and marker publishers must also be running.

Check incoming estimates with:

```
bash
```

```
ros2 topic echo /megapose/object_pose
```

Your current `ros2 launch mpose_live demo.launch.py` starts **only the bridge**; the error node is commented out.

If you mean the **TurtleBot controller**, that file and its executable entry point are still absent from the package, so it cannot be launched yet.

096 You

Sep 28, 2026 04:51 PM EDT

`rt_live` publishes data to `mpose_bridge` via udp socket. Can't I write a `ros2` node for `rt_live` for it to directly publish it's pose data as `ros2` topic ?

097 Assistant • progress update

Sep 28, 2026 04:56 PM EDT

Yes—a ROS 2 node can capture the images, run MegaPose, and publish poses directly. I'll check the current Python environments, since their compatibility was the reason for using UDP.

098 Assistant

Sep 28, 2026 04:57 PM EDT

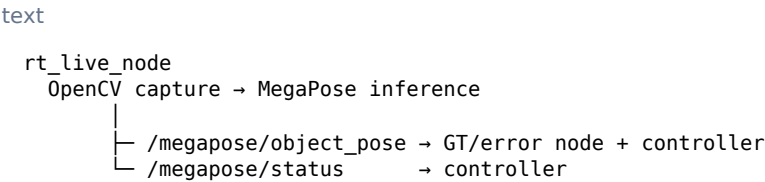
Yes. `rt_live.py` **can become a ROS 2 node and publish poses directly.** UDP was a workaround for your Python environments.

I checked them again:

Environment	Python	Current situation
mpose	3.9.23	MegaPose environment; importing rclpy fails
ROS 2 Humble	3.10.12	rclpy works; Torch and MegaPose are not installed

ROS’s compiled Python extensions must match the interpreter. Sourcing ROS inside the current Python 3.9 environment cannot fix that incompatibility. [Official ROS documentation](#)

The direct architecture would be:



To implement it:

1. Create a separate **system-Python 3.10 virtual environment** with access to ROS packages.
2. Install and validate MegaPose’s dependencies there, including Torch and its renderer.
3. Wrap the existing capture/inference code in an rclpy node. Replace `sender.send(...)` with ROS publishers while preserving the image timestamp and tracking status.

Then `mpose_bridge_node` **is unnecessary**. The GT/error node and controller can keep using the same topics.

The main work is making MegaPose run in the ROS-compatible environment; publishing its output as PoseStamped is straightforward. The GPU driver issue also needs to be resolved before testing live inference.

099 You

Sep 28, 2026 05:14 PM EDT

importing megapose into a 3.10 environemnt may not work.

100 Assistant • progress update

Sep 28, 2026 05:14 PM EDT

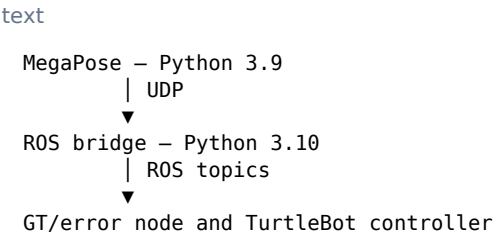
That’s a valid concern—I haven’t verified MegaPose on Python 3.10. I’ll check its dependency files before recommending that migration.

101 Assistant

Sep 28, 2026 05:14 PM EDT

Yes—**Python 3.10 compatibility is unverified**, and I shouldn’t have presented migration as straightforward. Your MegaPose checkout explicitly pins Python 3.9 in `environment.yaml` (`/home/abhi/dev/megapose6d/conda/environment.yaml:8`).

For this demo, keep the working separation:



A ROS node cannot directly import MegaPose from a separate Python 3.9 environment. It could **launch** `rt_live.py` **as a subprocess**, but communication between the processes would still be needed.

If the inconvenience is starting everything separately, **one ROS launch file can start** `rt_live.py` **using the `mpose` interpreter and start the ROS nodes together**. That simplifies operation while preserving the working environments and UDP connection.

102 You

Sep 28, 2026 05:38 PM EDT

Please export this entire chat into a pdf